Zipher Binary Communications Specification

Abstract

This document describes the binary communications protocol that can be used with Zipher machines. The protocol may be used over differing transport layers.

Revision History

Revision	Author	Date	Notes
1.00	Phil Hart	19th September 2000	Original
1.01	Phil Hart	17th November 2000	Amendments after initial feedback
1.02	Phil Hart	6th December 2000	Corrections to text, added efficiencies to CTS, Short Error Info, Batch Completed to Job Info and Current Time/Date.
1.03	Scott Langham	23rd February 2001	Added the Protocol Error: RFF_COMMANDNOTACCEPTED
1.04	Phil Hart	6th March 2001	Added BMX and EMX and FTC messages, documented Job and Multiple job download procedure. Additional ERR types, added Clarity update file type.
1.05	Phil Hart	23rd January 2002	Amended Flags for the SEL message to include JF_USECURRENTDATA and JF_NOBATCHCOUNTRESET
1.06	Phil Hart	19th March 2002	Added CMV message to determine the version of CLARiTY communications. This is version 1 of the communications protocol.
1.07	Phil Hart	28th June 2002	Added the `JF_CLEARLASTPRODLIMIT` flag to the product selection message (SEL)
1.08	Daniel Thompson	1st October 2002	Added CF_JOBNAME flag to CIFF File type specific flags section of the FRQ message.
2.00	Phil Hart	25th October 2002	Added the Host Communications Version (HCV) for backwards compatibility support, revised the FRQ message content, and updated the CMV tag. This is version 2 of the communications protocol. Added more AIU information.
3.00	Phil Hart	8th May 2003	This is version 3 of the communications protocol. Added CLE command. Added Appendix A to list difference between v2 and v3.
4.00	Ben Yeomans	3rd November 2003	This is version 4 of the communications protocol. Refer to Appendix A for differences between v3 and v4.
4.01	Phil Hart	19th April 2004	Added support for directory listings of log files and onboard image editing.
5.00	Phil Hart	28th January 2005	This is version 5 of the communications protocol. Refer to Appendix A for differences between v4 and v5.
5.01	Ben Yeomans	29th July 2005	Removed references to unimplemented features from the documentation.
7.00	Matthew Faithfull	07th November 2005	This is version 7 of the communications protocol. Refer to Appendix A for differences between v5 and v7.
7.01	Phil Hart	1st March 2006	Added FB_NOMSMODE to the CMV tag of the IRP message, added a few extra ERR Reasons for Failure
7.02	Phil Hart	15th August 2006	Added FB_SCRIPTS to the CMV tag
7.03	Phil Hart	8th October 2007	Added support for temporary bitmaps and DLC_TEMPGRAPHICS in directory listing.
8.00	Phil Hart	17th October 2007	This is version 8 of the communication protocol. Added optimisations for TCP/IP connections. Added random imaging section to common operations

9.00	Phil Hart	14th November 2007	This is version 9 of the communication protocol. Added the following messages FTD, LSL. Added support for line selection mode. Added JF_EXPIREDPRINTLIMIT flag to SEL.
9.01	Phil Hart	19th February 2008	Added extra file types for file transfer messages.
9.02	Phil Hart	3rd March 2008	Added confirmed print message (CPR).
9.03	Phil Hart	10th June 2008	Added FB_SLAVEUICONFIG to the CMV tag.
9.04	Phil Hart	2nd December 2008	Added JF_RESETIMAGESEQUENCE flag to JDU.
10.00	Phil Hart	25th February 2009	This is version 10 of the communication protocol. Added FB_SERIAL and FB_TCPIP to CMV Tag. Added the JF_AUTOCLEARLIMITFAULT flag to SEL, JDU. FNF changes
10.01	Phil Hart	17th May 2010	Added FMP command ID to the CMD message.
10.02	Kerri Brown	2nd Aug 2011	Added ribbon <totalusage> tag to SUP message.
11.00	Phil Hart	2nd Aug 2012	Added temporary CIFF file to file types and job selection commands.
11.01	Jeff Froggatt	31st October 2013	Added commands to the CMD table used by the Bark (8610) project.
11.02	Kerri Brown	2nd January 2014	Corrected spelling mistakes.
11.03	Rowan Eyres	30th October 2014	Added that ACK and NAK packets should also mirror Asynchronous bits and Transaction ID.
11.04	Jude Dalton	26th March 2015	Added scanner config and APL file entries for FRQ directory listing.
11.05	Jeff Froggatt	15th July 2015	Corrected the example for the STS 0x0001 message.
11.06	Jeff Froggatt	22nd July 2015	Corrected the example for the JIN 0x0003 message.
11.07	Daniel Varley	19th August 2015	Added SHD, SHO, SDO, SID, SMR, SCF and SIR commands for serialisation. Added DFS tag to IRP command.
11.08	Daniel Varley	28th August 2015	Updated SIR tags SNI, SLI & SRC.
11.09	Daniel Varley	28th October 2015	Changed SLI tag to SLR to be consistent with ZTC.
11.10	Rowan Eyres	15th July 2016	Updated the Feature Bitmask table for CSFREEVERSION, APLFILES, SCANNERCONFIGS, MULTIMACHINEMODE, SHIFTEXCCEPTIONS and SERIALISATION
11.11	Daniel Varley	24th August 2017	Updated IRQ SUP command tag to include 8520 cartridge information and example.
11.12	Jeff Froggatt	30th April 2018	Updated with ZJF file support and IAssure file support.
11.13	Jeff Froggatt	11th March 2019	Updated to include iAssure directory listing
11.14	Jonathan Sugden	7th October 2019	Added IAS (iAssure Information) tag to IRQ, AIS_IASSURE_CAPTURE_READY asynchronous tag to AIS and AIR messages, and BGF_CURRENTIASSURE bitmap download flag.
11.15	Aniebiet Akpan	24th May 2021	Updated DEL command to Delete All Jobs.
11.16	Phil Hart	13th August 2021	Added the Pallet operations to the CMD command.
11.17	Phil Hart	16th December 2022	Added the Part Pallet commands to the CMD command.
11.18	Phil Hart	7th February 2023	Added the Pallet Abort command to the CMD command.

Table of Contents

Abstract 1
Revision History 2
Table of Contents 4
Introduction 7
Conventions Used in this Document 7
Packet Framing (RS232 and TCP/IP) 8
Packet Framing 8
Start Character 8
Packet Flags 8
Packet Size 9
Transaction ID 9
Sequence ID 9
Address Information 10
End Character 10
Header Checksum 10
Packet Data 10
Data Checksum 10
Notes 11
Packet Protocol 12
Single Packet Transfers 12
Multi-Packet Transfers 12
Binary Communication Protocol 14
Message Header 14
Supported Message Types 14
No Data to Report (NUL – 0x0000) 16
Sample Message 16
Information Request (IRQ – 0x5001) 17
Number of Tags 17
Tag IDs 17
Sample Message 17
Information Response (IRP – 0x5002) 18
Information Tag IDs 18
State and Status Information (STS – 0x0001) 19
Sample State and Status Information Tag 20
Counts & Performance Information (CTS – 0x0002) 20
Sample Counts & Performance Information Tag 20
Job Information (JIN – 0x0003) 20
Sample Job Information Tag 21
Machine Information (MCH – 0x0004) 21
Sample Machine Information Tag 21
Current Time (TIM – 0x0006) 21
Sample Current Time Tag 22
Communications Version (CMV – 0x0007) 22
Sample Communications Version Tag 24
Job Information Unicode (JIU – 0x0008) 24
Determine Free Space (DFS – 0x0009) 24
Error Information (ERR – 0x1001) 24
Supplies Information (SUP – 0x1005) 25
Version Information (VER – 0x1006) 26
Limited Error Information (LEI – 0x1009) 27
iAssure Information (IAS – 0x100A) 28
File Request (FRQ – 0x5003) 29
Directory Listing 30
CIFF File 31
CIFF Data File 32
Bitmap Graphic 32
Temporary CIFF File 33

Temporary Bitmap Graphic 34
Sample Message 34
File Request Unicode (FRU – 0x5032) 35
File Transmit (FTX – 0x5004) 36
Sample Message 36
File Transmit Unicode (FTU – 0x5033) 37
File Transmit Unicode with Date (FTD – 0x503A) 38
File Not Found (FNF – 0x5005) 40
Sample Message 40
Poll for Requests (PFR – 0x5006) 41
Sample Message 41
Select Job (SEL – 0x5007) 42
Sample Message 44
Select Job Unicode (SLU – 0x5030) 45
Job Data Update (JDU – 0x5008) 46
Sample Message 47
Job Update Confirmation (JUC – 0x5022) 48
Multiple Job Data Update (MJU – 0x503D) 49
Sample Message 50
Advise on Information Update (AIU – 0x5009) 51
Sample Message 51
Asynchronous Information Subscription (AIS – 0x5035) 52
Sample Message 54
Asynchronous Information Response (AIR – 0x5036) 55
Job Select (AIS_JOBSELECT – 0x0000) 55
Transmitted Field (AIS_TRANSMITFIELD – 0x0001) 56
Printer Is Online (AIS_PRINTER_IS_ONLINE – 0x0002) 56
Printer Enters Warning (AIS_PRINTER_ENTERS_WARNING – 0x0003) 56
Printer Is Busy (AIS_PRINTER_IS_BUSY- 0x0004) 56
Printer Starts Printing (AIS_PRINTER_STARTS_PRINTING – 0x0005) 57
Labeller May Feed (AIS_LABELLER_MAY_FEED – 0x0006) 57
Print Cycle Starts (AIS_PRINT_CYCLE_STARTS – 0x0007) 57
Printer Is Offline (AIS_PRINTER_IS_OFFLINE – 0x0008) 57
Printer Leaves Warning (AIS_PRINTER_LEAVES_WARNING – 0x0009) 57
Printer Is Not Busy (AIS_PRINTER_IS_NOT_BUSY – 0x000A) 58
Printer Finishes Printing (AIS_PRINTER_FINISHES_PRINTING – 0x000B) 58
Labeller May Not Feed (AIS_LABELLER_MAY_NOT_FEED – 0x000C) 58
Print Cycle Completes (AIS_PRINT_CYCLE_COMPLETES – 0x000D) 58
Job Update Queue Is Full (AIS_JOB_UPDATE_Q_IS_FULL – 0x000E) 59
Job Update Queue Is Not Full (AIS_JOB_UPDATE_Q_IS_NOT_FULL – 0x000F) 59
New Allocation Received (AIS_NEW_ALLOCATION_RECEIVED – 0x0010) 59
Job Allocation Complete (AIS_JOB_ALLOCATION_COMPLETE – 0x0011) 59
Print Failed (AIS_PRINT_FAILED – 0x0012) 59
Printer Leaves Fault (AIS_PRINTER_LEAVES_FAULT – 0x0013) 60
Printer Enter Fault (AIS_PRINTER_ENTERS_FAULT – 0x0014) 60
Print Signal Ignored (AIS_PRINT_SIGNAL_IGNORED – 0x0015) 60
Rendering Complete (AIS_RENDERING_COMPLETE – 0x0016) 60
Rendering Failed (AIS_RENDERING_FAILED – 0x0017) 61
Job Update Queue Is Empty (AIS_JOB_UPDATE_Q_IS_EMPTY – 0x0018) 61
Job Update Queue Is Not Empty (AIS_JOB_UPDATE_Q_IS_NOT_EMPTY – 0x0019) 61
Job Allocation Aborted (AIS_JOB_ALLOCATION_ABORTED – 0x001A) 61
Job Update Queue Is Low (AIS_JOB_UPDATE_Q_IS_LOW – 0x001B) 61
Job Update Queue Is High (AIS_JOB_UPDATE_Q_IS_HIGH – 0x001C) 62
iAssure Capture Ready (AIS_IASSURE_CAPTURE_READY – 0x001D) 62
Command (CMD – 0x500A) 63
Sample Message 64
Delete File (DEL – 0x500B) 65
Sample Message 65
Delete File Unicode (DLU – 0x5031) 66

Delete All Jobs (DEL – 0x500B) ... 66
Sample Message ... 66
Protocol Error Message (ERR – 0x500D) ... 67
Sample Message ... 68
Set Time and Date (CLK – 0x500E) ... 69
Sample Message ... 69
File Transfer Complete (FTC – 0x500F) ... 70
Sample Message ... 70
Begin Multiple File Transfer (BMX – 0x5010) ... 71
Sample Message ... 71
End Multiple File Transfer (EMX – 0x5011) ... 72
Sample Message ... 72
Host Communication Version (HCV – 0x5020) ... 73
Sample Message ... 73
Clear Error Messages (CEM – 0x5021) ... 74
Sample Message ... 75
Erase Queue Item (EQI – 0x5037) ... 76
Erase Queue Item Response (EQR – 0x5038) ... 77
Line Select (LSL – 0x503B) ... 78
Sample Message ... 78
Confirmed Print (CPR – 0x503C) ... 79
Sample Message ... 79
Serialisation Information Request (SIR – 0x503E) ... 80
Number of Tags ... 80
Tag IDs ... 80
Sample Message ... 80
Serialisation Information Response (SRP – 0x503F) ... 81
Serialisation Information Tag IDs ... 81
Serialisation Record Count (SRC – 0x0001) ... 81
Serialisation Next Record Index (SNI – 0x0002) ... 82
Serialisation Last Record Index (SLR – 0x0003) ... 82
Serialisation Get Maximum Records (SGM – 0x0004) ... 82
Serialisation Get Free Space (SFS – 0x0005) ... 82
Serialisation Clear Buffer (SCB – 0x0006) ... 83
Serialisation Header Only (SHO – 0x5040) ... 83
Sample Message ... 83
Serialisation Data Only (SDO – 0x5041) ... 83
Sample Message ... 84
Serialisation Indexed Data (SID – 0x5042) ... 84
Sample Message ... 85
Serialisation Header & Data (SHD – 0x5043) ... 85
Sample Message ... 85
Serialisation Set Max Records (SMR – 0x5044) ... 86
Sample Message ... 86
Serialisation Change Field Data (SCF – 0x5045) ... 86
Sample Message ... 87
Common Operations ... 88
Standard Polling Loop ... 88
Single Job Downloads ... 88
File Dependencies ... 89
Multiple Job Downloads ... 89
Using the AIU and AIS Messages ... 91
Message Collision ... 91
Using the AIS message over a TCP/IP connection ... 91
Random Imaging Applications ... 92
Tips: ... 92
A typical random imaging application ... 92
Appendix A: Differences in message behaviour ... 94

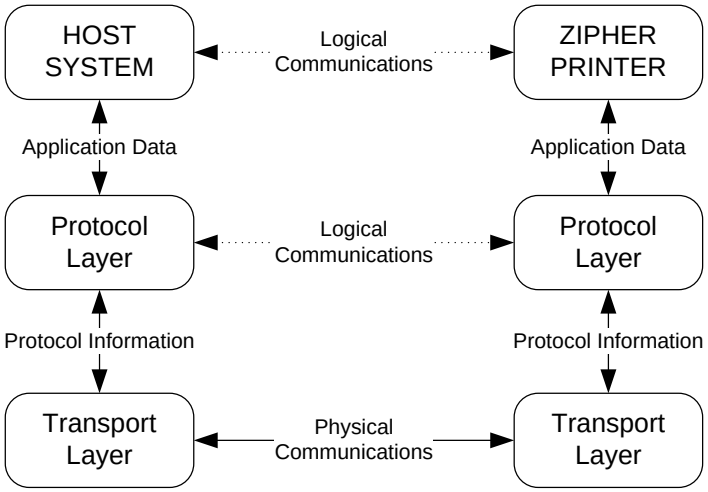
Introduction

This document describes the binary communications protocol and packet framing that can be used to communicate with a Zipher printer. The communications protocol is identical for all methods of transmission (RS232, TCP/IP etc.), however the framing of the packets may vary. The first section of this specification describes the various framing mechanisms for the support transport protocols. The second section describes the content of the messages that may be sent, and the responses that could be received.

Initially, Zipher machines will support RS232/RS485 and TCP/IP connections. All connections are point to point, with no support for multi-drop RS485 or RS422. The protocol is master-slave with the host machine considered as the master, and responsible for initiating any communications. The master can instruct the Zipher machine to send out messages asynchronously when various status changes occur. For this to function correctly, full duplex communications need to be supported.

The communications between the host system and the Zipher printer is split logically into two halves. The lower layer is the framing of the messages, and is dependent upon the transmission mechanism. Wherever possible standard transmission protocols will be used (TCP/IP etc.). However, for serial communications a Zipher specific system has been implemented. The transmission layer is concerned with ensuring that the protocol data is delivered to the device correctly, and splitting up messages that are larger than the maximum packet size into multiple packets.

The second layer is protocol layer, which is completely independent of the transmission medium. It contains a number of commands, requests and responses. It also indicates what responses are valid for any command or request given the context of the communication.



Conventions Used in this Document

All character values described in this document are binary values. This means that if the specification states that the value 65 should be sent; then ASCII character 65 ('A') needs to be sent, not the ASCII characters for 6 and 5. The characters '0x' precede values that are stated in hexadecimal. So the value 0x41 means that the value 65 should be sent.

All values that are larger than 1 byte will be sent least significant byte first (Intel byte order). So if the decimal value 30000 (hexadecimal 0x7530) needed to be sent as a 2-byte value, this should be sent as the bytes 0x30 followed by 0x75 (or 48 followed by 117).

Packet Framing (RS232 and TCP/IP)

The transport layer, and framing of information for the packet layer is Zipher specific. It deals with ensuring correct delivery of messages using checksums and a retry mechanism. It also deals with splitting up a message that is larger than the maximum packet size into multiple blocks.

Packet Framing

The framing of the packet is a follows:

Name	Size	Description
Start Character	1 Byte	1 Byte start of message character will always contain the value 0xA5
Packet Flags	1 Byte	Flags indicating the format and content of this packet
Packet Size	2 Byte	Size of the packet including all of the framing information and checksum
Transaction ID	2 Byte	The transaction ID of the packet. See below.
Sequence ID	2 Byte	Sequence Identifier. If this message is part of a multi-packet transfer this contains the ID of the packet
*Address Information	4 Bytes	Reserved – if the ADDR flag is set in the Packet flags 4 bytes will be allowed between the sequence ID and the End Character.
End Character	1 Byte	1 byte end of header character will always contain the value 0xE4
Header Checksum	1 Byte	Sum of all of the bytes in the header in to a single byte and then inverted.
Packet Data	Up to 2000 Bytes	Data content of the message
*Data Checksum	2 Bytes	Optional 2 bytes

*Optional

Start Character

The Start Character is a fixed value that is used to synchronise the start of a message. The receiver should not begin to process the packet framing until the value 0xA5 has been received.

Packet Flags

The packet flags indicate the type, structure and formatting of the packet that is about to be received. The flags are transmitted as a packed byte in the following format.

Bit Number

7	6	5	4	3	2	1	0
RES	ASY	ADR	CS	NAK	ACK	FIN	SQS

Bit Number	Abbreviation	Name
0	SQS	Sequence Start
1	FIN	Sequence Finish
2	ACK	Acknowledgement
3	NAK	Negative Acknowledgement
4	CS	Checksum Present
5	ADR	Address Information Present
6	ASY	Asynchronous Message

7	RES	Reserved

The SQS flag indicates the start of a sequence of packets that makes up the full message. It should be set at the start of any new message transmission, even if the message requires only one packet to transmit.

The FIN flag indicates the end of a sequence of packets that make up a full message. It should be set at the end of any message transmission. In the case that there is only one packet required for a message, both the SQS and FIN flags must be set.

The ACK flag indicates that this message is an acknowledgement of correct receipt of the previous message. A message with the ACK flag set contains no data payload.

The NAK flag indicates that this message is a negative acknowledgement of receipt of the previous message. This means that the message was not received correctly. This may occur if the packet size exceeds the maximum allowable, the sequence ID is invalid, or the checksum data is incorrect.

The CS flag indicates that this packet will be followed by a 2-byte checksum at the end of the data section. If this flag is not set, then no checksum is required. The header checksum will be validated if the bit is set, and ignored if the value is clear. See the checksum section for more information on calculating checksums.

The ADR flag indicates that 4 bytes of additional address information will be present in the packet. This will be sited between the Sequence ID and the packet data.

The ASY flag indicates that this is an Asynchronous message sent by the slave (the Zipher printer). Asynchronous messages will only occur when the master registers with the printer as requiring notification when certain events happen. This can be used for being informed when a print has occurred, or a new job has been selected.

The RES flag is reserved and **must** be set to 0. If this value is not 0, a NAK will be sent in response.

Packet Size

The packet size contains the total number of bytes that make up the packet. This includes all of the framing information, the packet data and any trailing checksum. The maximum size that any one packet can be is 2000 bytes for a serial connection. For TCPIP connections the maximum packet size is 65535 bytes for communication revisions greater than 3 (see appendix A). Any packets that are larger than this will be rejected the printer by means of a NAK message.

Transaction ID

The Transaction ID contains a unique value that is associated with the particular message. The transaction ID is set by the host and should be implemented as an incrementing count. The transaction ID should be incremented when the sequence finish bit is set. Special care should be taken when providing a transaction ID for the AIU message as this value will be used in all asynchronous responses. It is recommended that this value is guaranteed unique (i.e. uses the value 65535, and ensure that this value is never used for any other message type. The transaction ID is provided to allow the host to link outgoing messages to incoming, as the asynchronous messages can cause problem in this area.

Sequence ID

The Sequence ID contains the index of the message within a sequence. When the data that needs to be sent exceeds the maximum number of bytes that can be placed in one packet the sequence ID is used to index each packet. The sequence ID always starts at the value of 0 for the first packet and increases by 1 for each packet in the sequence. For messages that require only one packet, the

sequence ID must be set to 0. For more information on the use of Sequence Ids see the "Multi-Packet transfers" section.

Address Information

The address information for the packet is optional, and will only be expected if the ADR flag is set. Initially this information is not supported and should not be used. It is provided for future compatibility if a multi-drop network solution is required. For the moment, clients should not set the address information bit, and should ignore the 4 bytes in this position if a message is received with the address bit is set. If the address information bit is set, the four byte values should be set to 00 00 01 00.

End Character

The End Character is a fixed value that is used to synchronise the end of the header message. This is provided to ensure that the messages are not incorrectly detected.

Header Checksum

The header checksum validates that the header information is valid and not corrupt. It is calculated by simply summing each individual byte of the header into a byte value, any carry is simply lost. The final value is then inverted. If the checksum bit is not set to a 1 then this value is ignored, it is advised that this value is set to 0 in this case.

Packet Data

The packet data contains the actual packet data for the message. The content of this message is described in the protocol section of this document.

Data Checksum

The Checksum is used to ensure that the entire packet data received is identical to the data transmitted. The checksum should be calculated on the entirety of the packet data starting with the first character of the packet data and ending with the last character of the packet data. If no packet data is transmitted (i.e. this is an ACK packet) then no checksum is expected or will be sent. The checksum is a 16-bit cyclic redundancy check and sample C source code for this is presented below.

```
typedef union
{
        WORD word;
        struct
        {
                BYTE lo;
                BYTE hi;
        }byte;
} LO_HI;

// Calculate the CRC value for byte value i.
WORD crc_table(unsigned char i)
{
        WORD j;
        WORD k;
        WORD crc;

        k = i << 8;
        crc = 0;
        for( j = 0; j < 8; j++ )
```

```
        {
                if( ( crc ^ k ) & 0x8000) crc = (crc << 1) ^ 0x1021;
                else crc <<= 1;
                k <<= 1;
        }
        return crc;
}

//This is an entry point for external calls
//Define this function in the header file
unsigned int count_crc(PBYTE *ptr, WORD count)
{
        LO_HI crc, temp;
        WORD  i;

        crc.word = 0;
        temp.word = 0;
        for (i=0;i<count;i++)
        {
                temp.word = crc_table(crc.byte.hi);
                crc.byte.hi = temp.byte.hi ^ crc.byte.lo;
                crc.byte.lo = temp.byte.lo ^ ptr[i];
        }
        return crc.word;
}
```

Notes

The machine will always respond to the host with the same packet structure as the last message received from the host. So if the host uses checksums and address information, the machine will respond with the same format.

By default, the machine uses checksums and address information. The address information for an outgoing message is set to 01 00 00 00.

Packet Protocol

The Serial Communications transport layer is intended to be completely transparent to the Binary Protocol layer. This means that the Binary Protocol should have no knowledge, or require any information about the packetisation of the data for transmission. It is intended that the binary protocol data is passed to the transport layer as a single block of data, the transport layer will packetise this as it sees fit, and then transmit it. It will wait for a response and when a complete response has been received, pass this back to the protocol layer.

The packet protocol is a master-slave protocol, and at the start of any communication, the host is considered to be the master. The master sends out the first packet of data any waits for a response. The response should be received within 2 seconds depending upon the latencies of the transmission medium. Every packet of information that is transmitted by the transport layer will be responded to by either an ACK or NAK packet. This will continue until the last packet of the message is transmitted (i.e. it has the FIN bit set) and the ACK for this packet has been received. At this point the mastership changes hands to the Zipher Printer, which will send a response to the message in the same manner.

ACK and NAK packets are special packets that contain no data payload. They should echo back the state of the SQS, FIN and Asynchronous bits and the Sequence ID and Transaction ID values. The example below shows an outgoing packet being responded to by an ACK message. The packet illustrated from the host is the start of a multi-packet transfer as only the SQS bit is set. The checksum and packet data are not shown.

```
HOST>       0xA5 0x11 0x08 0x02 0x00 0x00 …… 0xXX 0xXX
ZIPHER<     0xA5 0x15 0x08 0x00 0x00 0x00 0xXX 0xXX
```

When the slave responds with an ACK the master should send the next packet in the sequence. Packets must be sent in sequence, with the sequence it incrementing each time. If the slave responds with an NAK message, the master should resend the previous message again. The master should attempt to retry 5 times before aborting the transfer. The Zipher printer will reset its sequence counts, and abort any active transfer when 5 seconds of inactivity or incorrect packets are received. The Zipher Printer will also automatically abort any active sequence if it receives a new packet with the SQS bit set.

Single Packet Transfers

Single Packet transfers are considered to be special cases of multi-packet transfers. A single packet transfer has both the SQS and FIN bits set in its flags, and has its Sequence ID set to 0. A simple illustration of this is set out below, here the HOST sends out a single packet transfer to the Zipher printer, which ACKs, then responds with its own single packet transfer, which the Host ACKs.

```
HOST>       0xA5 0x13 0x18 0x00 0x00 0x00 …… 0xXX 0xXX
ZIPHER<     0xA5 0x17 0x08 0x00 0x00 0x00 0xXX 0xXX

ZIPHER<     0xA5 0x13 0x48 0x00 0x00 0x00 …… 0xXX 0xXX
HOST>       0xA5 0x17 0x08 0x00 0x00 0x00 0xXX 0xXX
```

This example demonstrates the protocol in its most basic form, and would be used when polling for status of the printer. In this scenario, the Host sends out a request for the printer status (the first two lines of the example), and then the printer responds with its status.

Multi-Packet Transfers

Multiple packet transfers are only required when the size of the data that needs to be sent exceeds 2000 bytes (the maximum packet size). The first packet of a multiple packet transfer will have its

SQS set and the Sequence ID will be set to 0. All of the following data packets will add one to the Sequence ID, and the SQS/FIN bits will not be set. The final packet of the sequence will have the FIN bit set.

All of the packets except the last one should contain the maximum packet size number of bytes (2000) in their data payload. The packet of the transfer is variable length between 1 byte and the maximum packet size.

The following example illustrates a 3-packet transfer of a message of 1200 bytes in size. The Zipher printer responds with a 2-packet response of 812 bytes.

… Host is the master and transmits a message of 1200 bytes …

```
HOST>       0xA5 0x11 0x08 0x02 0x00 0x00 …… 0xXX 0xXX
ZIPHER<     0xA5 0x15 0x08 0x00 0x00 0x00 0xXX 0xXX
HOST>       0xA5 0x10 0x08 0x02 0x01 0x00 …… 0xXX 0xXX
ZIPHER<     0xA5 0x14 0x08 0x00 0x01 0x00 0xXX 0xXX
HOST>       0xA5 0x12 0xB8 0x00 0x02 0x00 …… 0xXX 0xXX
ZIPHER<     0xA5 0x16 0x08 0x00 0x02 0x00 0xXX 0xXX
```

… Zipher printer becomes the master and transmits a message of 812 bytes …

```
ZIPHER<     0xA5 0x11 0x08 0x02 0x00 0x00 …… 0xXX 0xXX
HOST>       0xA5 0x15 0x08 0x00 0x00 0x00 0xXX 0xXX
ZIPHER<     0xA5 0x12 0x34 0x01 0x01 0x00 …… 0xXX 0xXX
HOST>       0xA5 0x16 0x08 0x00 0x01 0x00 0xXX 0xXX
```

Binary Communication Protocol

The protocol layer of the binary communication protocol is also master-slave based. The host system sends out a command/request, and the Zipher printer will send back a response. The only exception to this occurs when 'advise' messages have been set up, as these can result in asynchronous messages being generated.

Every message begins with the same header, which consists of the message ID, and the message size. After this, the remainder of the data is message type specific. Consult the documentation of the message for information about the structure of the data.

Each message can only contain a single Message ID, however the tag system used on the information requests allows flexibility on what information is returned.

Message Header

The message header consists of:

Name	Size	Description
Message ID	2 Bytes	An identifier of the type of message received/transmitted.
Message Length	4 Bytes	The length of message data.

The Message ID is simply a number that identifies which command/response is contained within the message.

The Message Length is the total length of message data including the message ID and message length.

Supported Message Types

The table below lists all of the currently supported Message IDs.

Name	Value	Description
NUL	0x0000	No Data to report
IRQ	0x5001	Information Request
IRP	0x5002	Information Response
FRQ	0x5003	File Request
FTX	0x5004	File Transmit
FNF	0x5005	File Not Found
PFR	0x5006	Poll for Requests
SEL	0x5007	Select Job
JDU	0x5008	Job Data Update
AIU	0x5009	Advise on Information Update
CMD	0x500A	Command
DEL	0x500B	Delete File
ERR	0x500D	Protocol Error Message
CLK	0x500E	Set Time and Date
FTC	0x500F	File Transfer Complete
BMX	0x5010	Begin Multiple File Transfer
EMX	0x5011	End Multiple File Transfer
HCV	0x5020	Host Communication Version
CEM	0x5021	Clear Error Messages
JUC	0x5022	Job Update Confirmation
SLU	0x5030	Job Select Unicode

DLU	0x5031	Delete File Unicode
FRU	0x5032	File Request Unicode
FTU	0x5033	File Transmit Unicode
AIS	0x5035	Asynchronous Information Subscription
AIR	0x5036	Asynchronous Information Response
EQI	0x5037	Erase Queue Item
EQR	0x5038	Erase Queue Item Response
FTD	0x503A	Unicode File Transmit with Date
LSL	0x503B	Line Select
CPR	0x503C	Confirmed Print
SIR	0x503E	Serialisation Information Request
SRP	0x503F	Serialisation Information Response
SHO	0x5040	Serialisation Header Only
SDO	0x5041	Serialisation Data Only
SID	0x5042	Serialisation Indexed Data
SHD	0x5043	Serialisation Header and Data
SMR	0x5044	Serialisation Set Max Records
SCF	0x5045	Serialisation Change Field Data

No Data to Report (NUL – 0x0000)

The Zipher Printer returns this response when there is no data to report.

The only valid use of this message is as a response from the Zipher Printer to the Host machine. It is returned in response to a Poll For Request message when the Zipher printer has no requests that need to be reported.

The format of this message is as follows:

Name	Size	Description
Message ID	2 Bytes	0x0000
Message Length	4 Bytes	0x00000006

This response has no message body.

Sample Message

The following byte stream contains a No Data to Report message.

```
HOST> 0x00 0x00 0x06 0x00 0x00 0x00
```

Information Request (IRQ - 0x5001)

This request is used to extract various types of status information from the Zipher Printer. It can be used to extract performance information, job information, consumables status, warning and fault states, machine status, and barcode verification status from the machine.

The only valid use of this message is as a request from the Host machine to the Zipher Printer. The only acceptable response is an Information Response (IRP = 0x5002).

The format of this message is as follows:

Name	Size	Description
Message ID	2 Bytes	0x5001
Message Length	4 Bytes	Variable length >= 10
Number of Tags	2 Bytes	Number of Information Tags
Information Tag IDs	2 * n Bytes	Information Tag IDs of the information the host is interested in.

Number of Tags

This is a 2-byte value that indicates the number of information tags that follow this value. The value may be in the range 1 to 65535. Each Tag ID is a request for a specific set of information.

Tag IDs

The Tag ID is a 2-byte value that indicates a set of information that the Host is interested in. The complete list of supported information tags, and the data that is returned for them can be found under the Information Response message.

Sample Message

The following byte stream requests information tags 1 and 2.

```
HOST> 0x01 0x50 0x0C 0x00 0x00 0x00 0x02 0x00 0x01 0x00 0x02 0x00
```

Information Response (IRP – 0x5002)

This message provides information about the status of the Zipher Printer. The information that is returned is dependent upon the information requested in the Information Request (IRQ – 0x5001) message. The message uses a tag-based format to provide multiple blocks status information, based on the request.

The only valid use of this message is as a response from the Zipher Printer to the Host machine. It can be returned as a response to an Information Request, or indirectly as a result of the Advise on Information Update (AIU - 0x5009) message being set for a particular piece of status information. See the Advise on Information Update message for more details.

The format of this message is as follows:

Name	Size	Description
Message ID	2 Bytes	0x5002
Message Length	4 Bytes	Variable length > 12
Number of Tags	2 Bytes	Number of Information Tags
Information Tag ID	2 Bytes	Information Tag IDs of the information the host is interested in.
Information Tag Size	2 Bytes	Size of the data for this tag including ID and length
Information Tag Data	n Bytes	Tag dependant data, size given by the Tag Size – 4, structure of the data is given under each information tag description.

For each tag in the number of tags section there will be an Information Tag ID, and Information Tag Size and a block of tag data. The next tag information will immediately follow the end of the previous tag block.

Information Tag IDs

The following Information Tag IDs are supported:

Name	Value	Description
STS	0x0001	State and Status Information
CTS	0x0002	Counts & Performance
JIN	0x0003	Job Information
MCH	0x0004	Machine Information
TIM	0x0006	Current Time
CMV	0x0007	Communications Version
JIU	0x0008	Job Information Unicode
DFS	0x0009	Determine Free Space
ERR	0x1001	Error Information
SUP	0x1005	Supplies/Consumables Info
VER	0x1006	Version Information
LEI	0x1009	Limited Error Information
IAS	0x000A	iAssure Information

The value of the tag numbers indicates the type of data that will be returned. Tags IDs starting with 0 in their most significant nibble return pure binary data. Tags starting with 1 in their most significant nibble will return a XML data block.

NOTE: All tags are guaranteed to be an event length, and all multi-byte words will fall on an even byte boundary. Where necessary, tags will be padded with an extra NULL to ensure an even length. The Tag size will include this extra byte.

NOTE: The sizes quoted are not fixed, and may grow in the future. No information will be removed from a tag, but additional information may be appended to the end of the tag block. Any client software should take this into account.

The following section describes the content and format of each information tags.

State and Status Information (STS – 0x0001)

The state and status information is a binary tag that returns the overall state of the machine. It provides information on whether the machine is on/offline in error etc. It gives an indication of any actions that are active, or have just completed in the machine.

The format of the State and Status Information tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x0001
Information Tag Size	2 Bytes	16 + length of active error string
Machine State Flags	4 Bytes	A set of flags that indicate the current state of the machine.
Machine Action Flags	4 Bytes	A set of flags that indicate the current actions that the machine is performing, or has just completed
Active Error Type	2 Bytes	A value indicating the type of the highest priority error in the printer
Length of Error String	2 Bytes	The length of the error string including a terminating NULL – and any pad character required to make the string length even. Will be 0 if no error in the system.
Error String	n Bytes	The error id string for the highest priority error in the system. This is not intended to be human readable string – it is the equivalent of a windows string resource id.

The Machine state flags are defined as (see the Error Information tag section for more information on the differences between faults and warnings):

Name	Bit Number	Value	Description
MSF_PWRDWN	0	0x00000001	The machine has been powered/shut down – or has not power/started up
MSF_ONLINE	1	0x00000002	The machine is online
MSF_FAULT	2	0x00000004	The machine has a fault
MSF_WARNING	3	0x00000008	The machine has a warning
MSF_IMAGING	4	0x00000010	The machine is drawing the image for a new job, or is drawing the data for a JDU command

The machine action flags are defined as follows:

Name	Bit Number	Value	Description
MAF_NEWJOB	0	0x00000001	A new job has been selected, or the job data has been updated by the JDU command.

All of these flags are latched when the events occur, and reset when the status message is sent out. This means that the flags will only be seen in the next status report after the event occurs.
The values for the active error types are:

Name	Value	Description
AET_NOERROR	0x0000	The printer has no errors/faults/warnings
AET_WARNING	0x0001	The printer has at least one warnings
AET_FAULT	0x0002	The printer has at least one fault

The priority of these errors rises with value, so if the printer has a fault and a warning, the AET_FAULT value will be returned.

Sample State and Status Information Tag

A Zipher printer that is on-line with no errors, and a new job has been selected would return the following byte stream.

```
ZIPHER> 0x01 0x00 0x10 0x00 0x02 0x00 0x00 0x00 0x01 0x00 0x00 0x00 0x00
        0x00 0x00 0x00
```

Counts & Performance Information (CTS – 0x0002)

The counts and performance information is a binary tag that returns the number of prints for the current job, the total number of prints the machine has performed, and the current throughput.

The format of the Counts & Performance tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x0002
Information Tag Size	2 Bytes	0x000E
Batch Count	4 Bytes	The number of products that has been printed for the current job.
Total Count	4 Bytes	Total number of products that have been printed by the machine.
Packs Per Minute	2 Bytes	The current throughput of the machine.
Nominal Speed	2 Bytes	The target throughput of the machine.
Short Term Speed	2 Bytes	The short-term throughput of the machine.
Short Term Efficiency	2 Bytes	The short-term efficiency of the machine.
Long Term Speed	2 Bytes	The long-term throughput of the machine.
Long Term Efficiency	2 Bytes	The long-term throughput of the machine.
Trend Speed	2 Bytes	The trend throughput of the machine.
Trend Efficiency	2 Bytes	The trend throughput of the machine.

Sample Counts & Performance Information Tag

The following byte stream would be returned by a Zipher printer that had printed 1000 products on the current job, 6376 products overall and run at 30 packs per minute.

```
ZIPHER> 0x02 0x00 0x0E 0x00 0xE8 0x03 0x00 0x00 0xE8 0x18 0x00 0x00 0x1E
        0x00
```

Job Information (JIN – 0x0003)

The Job Information is a binary tag that returns the name of the current job, any limit to the number of prints for it, and the time and date it was selected, or last updated by a JDU message.

The format of the Job Information tags is:

Name	Size	Description

Information Tag ID	2 Bytes	0x0003
Information Tag Size	2 Bytes	0x????
Length of Job Name String	2 Bytes	The number of bytes in the job name string including the terminating NULL character and any pad character to make the length even.
Job Name String	n Bytes	The Name of the Currently selected Job with NULL termination.
Job Flags	4 Bytes	Reserved for future use. Set to zero.
Products Remaining	4 Bytes	Reserved for future use. Set to zero.
Selection Time (hours)	2 Bytes	The hour the job was selected (24 hr clock)
Selection Time (minutes)	2 Bytes	The minute the job was selected
Selection Time (seconds)	2 Bytes	The second the job was selected
Selection Date (day)	2 Bytes	The day of the month the job was selected
Selection Date (month)	2 Bytes	The month the job was selected
Selection Date (year)	2 Bytes	The year the job was selected (4 digits)

The selection time and date will also be updated by the Job Data Update command, which is considered to be the equivalent of a new Job selection.

Sample Job Information Tag

The following byte stream would be returned by a Zipher printer that had the Job "Strawberry" selected, and was selected at 10:12:44 17-Sept-2000.

```
ZIPHER> 0x03 0x00 0x24 0x00 0x0C 0x00 0x53 0x74 0x72 0x61 0x77 0x62 0x65
        0x72 0x72 0x79 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
        0x0A 0x00 0x0C 0x00 0x2C 0x00 0x11 0x00 0x09 0x00 0xD0 0x07
```

Machine Information (MCH – 0x0004)

The machine information is a binary tag that returns the name and model of the machine.

The format of the Machine Information tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x0004
Information Tag Size	2 Bytes	0x????
Length of Machine Name String	2 Bytes	The number of bytes in the Machine Name string including the terminating NULL character and any pad character to make the length even.
Machine Name String	n Bytes	The Name of Machine with NULL termination.
Length of Machine Model String	2 Bytes	The number of bytes in the Machine Model string including the terminating NULL character and any pad character to make the length even.
Machine Model String	n Bytes	The Model Name with NULL termination.

Sample Machine Information Tag

A Zipher Zodiac 53mm intermittent printer would return the following byte stream.

```
ZIPHER> 0x04 0x00 0x28 0x00 0x0E 0x00 0x5A 0x69 0x70 0x68 0x65 0x72 0x20
        0x5A 0x6F 0x64 0x69 0x61 0x63 0x00 0x12 0x00 0x35 0x33 0x6D 0x6D
        0x20 0x49 0x6E 0x74 0x65 0x72 0x6D 0x69 0x74 0x74 0x65 0x6E 0x74
        0x00
```

Current Time (TIM – 0x0006)

The current time is a binary tag that returns the current time and date at the printer. This value can be set via the CLK message.

The format of the Current Time tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x0006
Information Tag Size	2 Bytes	0x0010
Year Value	2 Bytes	The current year value (years from 1000 to 9999 can be sent).
Month Value	2 Bytes	The current month value (valid months are 1 to 12).
Day Value	2 Bytes	The current day value (values between 1 and 28 will be accepted, values between 29 and 31 are allowed if the month contains this many days).
Hour Value	2 Bytes	The current hour (value between 0 and 23 can be sent)
Minute Value	2 Bytes	The current minute (0-59)
Second Value	2 Bytes	The current second (0-59)

Sample Current Time Tag

A machine with the clock set to 14:33:00 on 15 Jan 2001 would return the following byte stream.

```
ZIPHER>  0x06 0x00 0x10 0x00 0xD1 0x07 0x01 0x00 0x0F 0x00 0x0E 0x00 0x21
         0x00 0x00 0x00
```

Communications Version (CMV – 0x0007)

Returns the version of the communications specification that the CLARiTY box supports. This can be used by the host to selectively enable transmission of certain messages. By enquiring the version of the communication standard before transmitting a message that is only supported by certain revisions of the CLARiTY software, the host can avoid having to wait for timeouts to expire.

The format of the Communications Version tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x0007
Information Tag Size	2 Bytes	0x000A
Comms Revision	2 Bytes	The revision of the communications specification.
Feature Bitmask	4 Bytes	Bitmask of the features supported by this version.

The comms revision indicates the revision of the specification that the CLARiTY supports. This message was only introduced at revision 1.06 of the communications specification. In CLARiTY software prior to that revision, nothing will be returned for this tag. The host should assume that revision of communications is 0 if this tag is not responded to.

The feature bitmask is used to indicate whether a particular communication/software feature is supported by this installation of CLARiTY.

The following table describes each revision of the communications specification and what functionality was added.

Revision	Functionality
1	Original. Added support for this message and 16 bit parameter transmission.

2	Added support for the host communications version message, remote database mode functionality and the additional data sections for the FRQ message.
4.00	Added windows printer driver support
4.01	Added FB_LOGFILES and FB_ONBOARDEDITOR feature flags.
5.03	Added FB_NOMSMODE flag.
5.05	Added FB_SCRIPTS flag.
7.03	Added

The bits defined for the Feature Bitmask are:

Name	Bit Number	Value	Revision Added	Description
FB_ 16BITPARAMS	0	0x00000001	1	Indicates that CLARiTY supports 16 parameter transmission
FB_HCVSUPPORT	1	0x00000002	2	Indicates that CLARiTY supports the HCV message
FB_REMOTEDB	2	0x00000004	2	Indicates that CLARiTY supports the remote database mode of operation
FB_WINPRINT	3	0x00000008	4.00	Indicates that CLARiTY supports the windows printer driver.
FB_LOGFILES	4	0x00000010	4.01	Indicates that CLARiTY supports retrieval of log files and log file directory listings.
FB_ONBOARDEDITOR	5	0x00000020	4.01	Indicates that CLARiTY supports the onboard image editor – and template CIFFs.
FB_NOMSMODE	6	0x00000040	5.03	Indicates that CLARiTY does not support Master Slave mode on this printer.
FB_SCRIPTS	7	0x00000080	5.04	Indicates that CLARiTY supports retrieval/download of script files and script file directory listings.
FB_RESTOREPOINTS	8	0x00000100	7.03	Indicates restore points are supported by the printer.
FB_SLAVEUICONFIG	9	0x00000200	9.03	Indicates that the printer supports uploading User Interface configuration files from a slave (master slave only).
FB_SERIAL	10	0x00000400	10.00	Indicates that the printer is connected via serial (for relayed connections e.g. MOXA boxes).
FB_TCPIP	11	0x00000800	10.00	Indicates that the printer is connected via TCP/IP (for relayed connections e.g. MOXA boxes).
FB_CSFREEVERSION	12	0x00001000	11.00	Currently Not Supported.
FB_APLFILES	13	0x00002000	11.04	Indicates that CLARiTY supports the transfer of APL files.
FB_SCANNERCONFIGS	14	0x00004000	11.04	Indicates that CLARiTY supports Scanner Configurations.
FB_MULTIMACHINEMODE	15	0x00008000	11.04	Indicates that CLARiTY supports Multi machine mode.
FB_SHIFTEXCEPTIONS	16	0x00010000	11.04	Indicates that CLARiTY supports Shift Exceptions.
FB_SERIALISATION	17	0x00020000	11.07	Indicates that CLARiTY supports Serialisation and the associated commands.

FB_IASSURE	18	0x00040000	11.12	Indicates that CLARiTY supports IAssure file transfer.
FB_ZJF	19	0x00080000	11.12	Indicates that CLARiTY supports the transfer of ZJF (Zipped Job Files).

Sample Communications Version Tag

A machine with the version 1 of the communications software and 16 bit parameter support would return the following byte stream.

```
ZIPHER>  0x07 0x00 0x0A 0x00 0x01 0x00 0x01 0x00 0x00 0x00
```

Job Information Unicode (JIU – 0x0008)

This tag is the same as the JIN tag except that the job name is specified in Unicode characters. It is available in protocol version 4 or later.

The format of the Job Information Unicode tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x0008
Information Tag Size	2 Bytes	0x????
Length of Job Name String	4 Bytes	The number of bytes in the job name string including the terminating NULL character.
Job Name String	n Bytes	The Name of the Currently selected Job in Unicode characters with NULL termination.
Job Flags	4 Bytes	Reserved for future use. Set to zero.
Products Remaining	4 Bytes	Reserved for future use. Set to zero.
Selection Time (hours)	2 Bytes	The hour the job was selected (24 hr clock)
Selection Time (minutes)	2 Bytes	The minute the job was selected
Selection Time (seconds)	2 Bytes	The second the job was selected
Selection Date (day)	2 Bytes	The day of the month the job was selected
Selection Date (month)	2 Bytes	The month the job was selected
Selection Date (year)	2 Bytes	The year the job was selected (4 digits)

The selection time and date will also be updated by the Job Data Update command, which is considered to be the equivalent of a new Job selection.

Determine Free Space (DFS – 0x0009)

This tag allows you to retrieve the amount of free space (in bytes) available on the Zipher Printer.

The format of the Determine Free Space tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x0009
Information Tag Size	2 Bytes	20
Total Space	8 Bytes	The total number of bytes on the Zipher Printer.
Free Space Available	8 Bytes	The amount of free space available in bytes.

Error Information (ERR – 0x1001)

The error information returns a string containing an XML block that describes the current faults and warnings.

Events that require user intervention are recorded and raised as an error, a fault or a warning depending upon the severity. A warning is the least critical event, and means that attention is not immediately required to rectify the problem and the print will continue to print. A fault is a more critical event that will stop the printer printing immediately. The user can only correct and clear a fault event if the problem that caused the fault has been corrected.

Name	Size	Description
Information Tag ID	2 Bytes	0x1001
Information Tag Size	2 Bytes	0x????
Length of XML Error String	2 Bytes	The number of bytes in the Error XML string including the terminating NULL character and any pad character to make the length even.
XML Error String	n Bytes	The XML Error String with NULL termination.

The XML block will take the following form:

```
<?xml version="1.0" encoding="UTF-8" ?>
<Errors>
        <Faults Total="2">
                <Error Name ="ERRINKOUT" Priority="0" CanClear="1" xml:space="preserve">
                        (E1) Error the ink has run out, place fresh bottle of ink in the unit to clear
                </Error>
                <Error Name="ERRFLTRBLK" Priority="1" CanClear="0" xml:space="preserve">
                        (E23) Error the main ink filter is blocked, replace the filter to clear
                </Error>
        </Faults>
        <Warnings Total="0" />
</Errors>
```

The total attribute of the faults and warnings tags contains the number of active faults and warnings. A value of 0 indicates that there are none. The name of the tag used for the faults and warnings is the Error ID as used in the State and Status tag. The priority attribute indicates which is the highest priority, with the lowest value being the highest priority. The CanClear attribute indicates that the problem has been fixed and can be cleared by the user or host. The text returned by the machine for the error will be in English.

Supplies Information (SUP - 0x1005)

The Supplies Information returns a string containing an XML block holding the status of the consumables within the machine.

Name	Size	Description
Information Tag ID	2 Bytes	0x1005
Information Tag Size	2 Bytes	0x????
Length of XML Supplies String	2 Bytes	The number of bytes in the XML Supplies string including the terminating NULL character and any pad character to make the length even.
XML Supplies String	n Bytes	The XML Supplies String with NULL termination.

The XML block will take the following form:

```
<?xml version="1.0" encoding="UTF-8" ?>
<Supplies>
        <Consumable type="Ink">
                <level state="Warning">35</level>
                <usage units="picolitres">50</usage>
                <expires>10:15:04 21/09/2000</expires>
                <lastchanged>15:35:43 18/09/2000</lastchanged>
        </Consumable>
```

```
<Consumable type=”Solvent”>
        <level state=”OK”>82</level>
        <usage units=”picolitres”>60</usage>
        <expires>18:12:04 24/09/2000</expires>
        <lastchanged>09:22:12 19/09/2000</lastchanged>
</Consumable>
<Consumable type=”Ribbon”>
        <level state=”OK”>82</level>
        <usage units=”metres”>60</usage>
        <expires>18:12:04 24/09/2000</expires>
        <lastchanged>09:22:12 19/09/2000</lastchanged>
        <totalusage>10</totalusage>
</Consumable>
<Consumable type=”Ribbon”>
        <level state=”OK”>82</level>
        <usage units=”metres”>60</usage>
        <expires>18:12:04 24/09/2000</expires>
        <lastchanged>09:22:12 19/09/2000</lastchanged>
        <totalusage>10</totalusage>
</Consumable>
<Consumable type="Ink-Cartridge1">
        <level state="OK">99</level>
        <usage>0</usage>
        <expires></expires>
        <lastchanged>08:20:45 09/08/2017</lastchanged>
        <inktype>Premium Black</inktype>
        <expirationdate>00:00:00 18/06/2019</expirationdate>
        <firstusedate>00:00:00 08/08/2017</firstusedate>
        <inkbatchnum>876543219DV</inkbatchnum>
        <cartridgeid>0604F53ACA88</cartridgeid>
</Consumable>

</Supplies>
```


The consumable element has the attribute “type” which contains a string describing the consumable type. The level element has the attribute “state” (OK|Warning|Critical) which indicates what the level value represents. The level value is the percentage level of the consumable, so 50 represent half of the consumable used. The usage value indicates the usage rate of the consumable every 1000 prints, and the units attribute gives the units that this value is in. The expires gives a time and date when the current consumable will run out assuming the current rate of usage. The lastchanged element gives the time and date that the consumable was last refreshed. All times and dates are in the format “hh:mm:ss dd/mm/yyyy”. The “Ribbon” consumable type has an additional <totalusage> tag which specifies the total ribbon usage in metres. The “Ink-CartridgeX” type on a 8520 printer will also return the ink type; expiration date of the cartridge; the first use date of the cartridge; the ink batch number and the cartridge ID.

Version Information (VER – 0x1006)

The Version Information returns a string containing an XML block with the versions of each of the components of the printer.

Name	Size	Description
Information Tag ID	2 Bytes	0x1006
Information Tag Size	2 Bytes	0x????
Length of XML Version String	2 Bytes	The number of bytes in the XML Version string including the terminating NULL character and any pad character to make the length even.
XML Version String	n Bytes	The XML Version String with NULL termination.

The XML block will take the following form:

```
<?xml version="1.0" encoding="UTF-8" ?>
<Version>
	<Object Name="Clarity">
		<Component Name="Build Date">
			02 JUN 2003
		</Component>
		<Component Name="Version">
			4.02.00 (2643)
		</Component>
		<Component Name="Part Number">
			401074.r13
		</Component>
		<Component Name="Operating System Build Date">
			2002-06-13 08:27:05
		</Component>
		<Component Name="Operating System">
			Microsoft Windows CE
		</Component>
		<Component Name="Operating System Version">
			3.0
		</Component>
		<Component Name="Platform">
			INTEL
		</Component>
	</Object>
	<Object Name="Inkjet Printhead 1">
		<Component Name="Altera Revision">
			01.05.00
		</Component>
		<Component Name="Build Date">
			25 MAR 2003
		</Component>
		<Component Name="Build String">
			1.06.00 (0182)
		</Component>
	</Object>
</Version>
```

The object tags define a section of the system; they contain a set of components. Each component has a name attribute, and contains a version number string. This message is provided mainly for automatic software update/versioning systems.

Limited Error Information (LEI - 0x1009)

The limited error information returns a string containing an XML block that describes the current faults and warnings.

Events that require user intervention are recorded and raised as an error, a fault or a warning depending upon the severity. A warning is the least critical event, and means that attention is not immediately required to rectify the problem and the print will continue to print. A fault is a more critical event that will stop the printer printing immediately. The user can only correct and clear a fault event if the problem that caused the fault has been corrected.

Name	Size	Description
Information Tag ID	2 Bytes	0x1001
Information Tag Size	2 Bytes	0x????

Length of XML Error String	2 Bytes	The number of bytes in the Error XML string including the terminating NULL character and any pad character to make the length even.
XML Error String	n Bytes	The XML Error String with NULL termination.

The XML block will take the following form:

```
<?xml version="1.0" encoding="UTF-8" ?>
<Errors>
        <Faults Total="2">
                <Error Name ="ERRINKOUT" Priority="0" CanClear="1" />
                <Error Name="ERRFLTRBLK" Priority="1" CanClear="0"/>
        </Faults>
        <Warnings Total="0" />
</Errors>
```

The total attribute of the faults and warnings tags contains the number of active faults and warnings. A value of 0 indicates that there are none. The name of the tag used for the faults and warnings is the Error ID as used in the State and Status tag. The priority attribute indicates which is the highest priority, with the lowest value being the highest priority. The CanClear attribute indicates that the problem has been fixed and can be cleared by the user or host.

iAssure Information (IAS - 0x100A)

The iAssure Information tag returns iAssure status information.

Name	Size	Description
Information Tag ID	2 Bytes	0x100A
Information Tag Size	2 Bytes	0x001A
iAssure Supported	1 Byte	Whether iAssure is supported. 0x1 indicates that iAssure is supported and 0x0 indicates that iAssure is not supported. If this value is 0, all subsequent data will consist of zeroes.
iAssure Enabled	1 Byte	Whether iAssure is currently enabled. 0x1 indicates that iAssure is enabled and 0x0 indicates that iAssure is disabled.
Total Captures	4 Bytes	The total recorded number of iAssure captures.
Failed Captures	4 Bytes	The total recorded number of captures which have been rejected by iAssure.
Last Failure Time (hours)	2 Bytes	The hour of the last rejected iAssure capture.
Last Failure Time (minutes)	2 Bytes	The minute of the last rejected iAssure capture.
Last Failure Time (seconds)	2 Bytes	The second of the last rejected iAssure capture.
Last Failure Date (day)	2 Bytes	The day of the last rejected iAssure capture.
Last Failure Date (month)	2 Bytes	The month of the last rejected iAssure capture.
Last Failure Date (year)	2 Bytes	The year of the last rejected iAssure capture.

File Request (FRQ – 0x5003)

This message allows a file to be requested from either the Zipher printer or from the Host machine. If the file exists on the recipient of the message, it will respond with a FTX message containing the file. If the file could not be found, then the recipient will respond with a FNF message.

The message can be used by either the Zipher Printer to request a file from the Host, or by the Host to request the file from the printer. The Host may transmit this message to the Zipher printer at any time. The Zipher printer will only transmit this to the host in response to a PFR message.

Some of the file types that can be requested have to be generated in response to the message passed (CIFF Data file, Directory Listings etc.). For these types of message, additional data may be specified to control the content of the files. For instance, the directory listing provides a list of all of the Jobs within the database of the printer. The printer supports sub-folders within its database to allow for Tiered job selection, by default if you ask for a job listing you are provided with the first level (or tier) of the database. The additional data allows lower tiers to be listed.

The format of this message is as follows:

Name	Size	Description
Message ID	2 Bytes	0x5003
Message Length	4 Bytes	Variable length > 10
File Type	2 Bytes	A value that identifies the type of file that is requested.
File Name String Length	2 Bytes	The length of the file name string, including a NULL terminator and any pad character to make the length even.
File Name String	n Bytes	The name of the file requested. This will not contain any path information, but may contain an extension.
Additional Data	n Bytes	Contains any optional data for the given file type. The file types that support optional data are listed below. If no optional data is specified for the given file type then no additional data section will be sent.

The defined values for the file types are:

Value	Description	Default Extension
0x0001	CIFF File (Job File)	.CIFF
0x0002	CIFF Data File	.XML
0x0003	Bitmap Graphic	.BMP
0x0004	True Type Font	.TTF/.TTC
0x0005	Parameter File	.XML
0x0006	Log File	.TXT
0x0007	Directory Listing	.XML
0x0008	Clarity Update File	None
0x0009	Current Parameters	.XML
0x000A	Template CIFF File	.TCIFF
0x000B	Comma Separated File	.CSV
0x000C	Encoder Profile File	.ENC
0x000D	Script File	.XML
0x000E	Restore Point File	
0x000F	Zipher Font File	.ZFBF
0x0010	Scanner Configuration File	.TXT
0x0011	Applicator Language File	.XML
0x0012	IAssure BMP File	.BMP

0x0013	IAssure ERB File	.ERB
0x0014	IAssure TXT File	.TXT
0x0015	ZJF File	.ZJF
0x8001	Temporary CIFF File	.CIFF
0x8003	Temporary Bitmap Graphic	.BMP

If an invalid file type or invalid file name is passed as part of this message, a File Not Found message will be returned. The filename passed does not require an extension, and if one is not present, the Zipher Printer will append the default one listed above. If an extension is provided, it should match the one in the table.

Clarity update files can have extensions of .DLL, .EXE, .XML and .H86. It is expected that the file transfer process maintain this extension.

File type specific behaviour:

Directory Listing

A directory listing allows the Host to retrieve a list of the CIFF files in the database of the printer. CLARiTY supports Tiered jobs which mean that the directory listing of the internal database of the printer may not be a single flat list, but could contain a number of sub-directories. The directory listing XML file is created in response to the request for the directory listing file, and will be filled with the data based on the arguments passed in the additional data. The format of the additional data for the directory listing is:

Name	Size	Description
Listing Content Flags	4 Bytes	Contains a number of bit flags determining the content of the listing see below
Sub-directory String Length	2 Bytes	The length of the sub-directory string, including a NULL terminator and any pad character to make the length even.
Sub-directory String	n Bytes	The name of the sub-directory to produce the listing of.

The Directory Listing type specific flags are defined as follows:

Name	Bit Number	Value	Description
DLC_SUBDIRECTORIES	0	0x00000001	Indicates that the listing should expand the content of subdirectories.
DLC_FONTS	1	0x00000002	When set the listing file contains only the fonts installed in the machine
DLC_LANGUAGES	2	0x00000004	When set the listing file contains only the languages installed in the machine (this flag will be ignored if DLC_FONTS is set).
DLC_LOGFILES	3	0x00000008	When set the listing file contains only the log files created by the machine.
DLC_TEMPLATES	4	0x00000010	When set the listing file contains only the template CIFF files installed in the machine.
DLC_ENTIREDATABASE	5	0x00000020	When set the listing file contains the entire contents of the printers' database (this includes jobs, graphics and fonts). The listing file includes file dates for synchronisation checking.
DLC_SCRIPTS	6	0x00000040	When set the listing file contains only the script files used by the machine.
DLC_RESTOREPOINTS	7	0x00000080	When set the listing file contains all system restore points

DLC_TEMPGRAPHICS	8	0x00000100	When set the listing file contains all temporary bitmap graphic files.
DLC_SCANNERCONFIGS	9	0x00000200	When set the listing file contains all the scanner configuration files used by the machine.
DLC_APLFILES	10	0x00000400	When set the listing file contains only the applicator programming language (APL) files used by the machine.
DLC_IASSUREFILES	11	0x00000800	When set the listing file contains only the iAssure files saved by the machine.
DLC_EXCLUDEFILESIZE	14	0x00004000	Indicates that file size information can be omitted from the Listing file. This significantly reduces the time required to create the listing file in large databases.
DLC_EXCLUDEMODIFYDATES	15	0x00008000	Indicates that date the file was modified or downloaded can be omitted from the Listing file. This significantly reduces the time required to create the listing file in large databases.

The default behaviour of the directory listing request is to return a listing of the root jobs directory, and not to expand the content of any of the sub-directories. There is no need to pass any additional data to the directory listing file request if the default behaviour is required.

The sub-directory string allows directory listings to be requested of directories beneath the root jobs directory. To request a listing of a directory called “150g” which is in the directory “Tesco” which is in the root directory, the string “Tesco/150g” should be passed as the sub-directory string. The root directory is identified as an empty string.

The format of the directory listing XML file is described in the “Directory Listing File Design Document” document.

CIFF File

The CIFF file type is used to transfer job (image) designs. The Additional Data section for the CIFF File type is used to provide context for the transfer. This allows a host to distinguish between a transfer that is required for a Job selection (where the content of the file may need to be looked up in a database), and a file transfer to fill the machines internal database (where the files may exist in a directory on the host).

The additional information for the CIFF Data File request is:

Name	Size	Description
File Content Flags	4 Bytes	Contains a number of bit flags determining the content of the CIFF Data File (see below)

The CIFF File type specific flags are defined as follows:

Name	Bit Number	Value	Description
CF_JOBNAME	0	0x00000001	Indicates that the file name given is the name of a job and may not actually be a file name. In Remote Database mode a slave may request a job from its master using a job name it obtained earlier from the master. In this situation the master will be responsible

	for resolving the job name into a file name prior to downloading the file.

CIFF Data File

The CIFF Data file can be used in one of two ways, firstly to change the user changeable (variable) fields within a Job during selection or when using the Job Data Update (JDU) message, and secondly it can be used as a list of the user changeable fields within a given job design.

When a CIFF Data file is requested from the printer the filename passed should be the name (and path if necessary) of the CIFF file within the job database of the printer. On receiving the request, CLARiTY will open up the CIFF file, extract from it all of the variable fields, and return a CIFF data file containing all of the user changeable fields. This allows the host to discover which fields within the Job are user changeable, before building a Select Job (SEL) message to select the Job.

The additional information for the CIFF Data File request is:

Name	Size	Description
File Content Flags	4 Bytes	Contains a number of bit flags determining the content of the CIFF Data File. This parameter is reserved for future use and should be set to zero.

Refer to the "CIFF Data File design document" for more details of the structure and content of this file.

Bitmap Graphic

The bitmap graphic file type can also be used to request a copy of the currently printing image (as displayed on the CLARiTY Controller). In order to achieve this, the following additional information needs to be supplied:

Name	Size	Description
Bitmap Graphic Flags	4 Bytes	Flags indicating the behaviour of this command. (see below)
Print head ID	2 Bytes	The ID of the print head to generate the image for.
Required Bitmap Width	2 Bytes	The width of the required bitmap in pixels. See below for an explanation of the calculation of the image size.
Required Bitmap Height	2 Bytes	The height of the required bitmap in pixels. See below for an explanation of the calculation of the image size.

The Bitmap Graphic type specific flags are defined as follows:

Name	Bit Number	Value	Description
BGF_CURRENTIMAGE	0	0x00000001	Indicates that a bitmap of the current image should be generated and sent to the host
BGF_MASPECT	1	0x00000002	Used in the calculation of image size. See below for an explanation.
BGF_DATAAREAONLY	2	0x00000004	Indicates that only the area of the image buffer containing data should be retrieved.
BGF_NODISPLAYRESSCALING	3	0x00000008	Indicates that any display resolution should be disabled (only valid if required width and

			height are 0).
BGF_FORMATGIF	8	0x00000100	Indicates that the bitmap should be retrieved in GIF format (the default format is BMP). Please see note below.
BGF_CURRENTIASSURE	9	0x00000200	Indicates that a bitmap of the current iAssure capture should be generated and returned to the host.

If the BGF_CURRENTIMAGE flag is not set (i.e. when this command is used to retrieve an existing file from disk) all other flags are ignored, as are the width, height and printhead id parameters.

When retrieving a bitmap of the currently selected job, you must specify the ID of the print head on which the image is printing. CLARiTY supports up to 4 separate print heads to be driven from a single controller. In the case of a single headed system, the value should be set to 1.

The bitmap of the currently selected job will be returned as a 2-colour bitmap of the size specified.

The BGF_CURRENTIASSURE flag, despite triggering an image download, does not support any other flags (such as BGF_MASPECT). In addition, if both the BGF_CURRENTIMAGE and BGF_CURRENTIASSURE flags are set, the BGF_CURRENTIMAGE flag will take priority and the BGF_CURRENTIASSURE flag will be ignored.

Calculation of image size

The calculation of the image size is explained in the table below. If the BGF_DATAAREAONLY flag is set, the term “image” refers to the area of the buffer containing data; otherwise it refers to the entire buffer.

Required Width	Required Height	BGF_MASPECT Flag	Result
0	0	Ignored	The image is not scaled and is retrieved in its original size. Display to print resolution scaling will be applied unless NODISPLAYRESSCALING is set.
> 0	0	Ignored	The image is scaled to the specified width and the height is calculated from the image’s aspect ratio.
0	> 0	Ignored	The image is scaled to the specified height and the width is calculated from the image’s aspect ratio.
> 0	> 0	Unset	The image is scaled to the exact width and height specified, even if it distorts the image.
> 0	> 0	Set	The image is scaled to be as large as possible within the specified width and height while maintaining the image’s aspect ratio.

GIF Format

The GIF files generated by this software are uncompressed GIF files which are compatible with the GIF standard but considerably larger than normal, compressed GIF files. For example, an image of width 4000 pixels and height 1500 pixels retrieved in GIF format will require over 3MB of storage space and will take a significant amount of time and processing to transfer, especially if a serial connection is being used. For this reason, the use of the GIF format is only recommended for images scaled to a small size. The BMP format provides more compact storage for large images.

The GIF encoding software is based in part on the work of the Independent JPEG Group. The Graphics Interchange Format(c) is the Copyright property of CompuServe Incorporated. GIF (sm) is a Service Mark property of CompuServe Incorporated.

Temporary CIFF File

Temporary CIFF files are CIFF files that are lost when the coder is powered off. They are identical to the CIFF Files above other than that they are stored in RAM and will be lost when the coder loses power. They are designed to be used in applications when a new image needs to be printed for each print, and host software will download a new CIFF to achieve this. They are automatically deleted from memory once they are selected. It is the responsibility of the host system to ensure that these CIFF files do not use up all of the coders RAM (there is approximately 16MB of RAM storage available). The host system should either delete the CIFF files if they are not selected or use a repeating pattern of names (e.g. 0.CIFF to 9.CIFF), to ensure that files are overwritten. Temporary CIFF files use RAM that is used by the CLARiTY Update system. It may be necessary to power a coder off and on that has a large number of temporary CIFF file installed in order to perform a CLARiTY Update.

Temporary Bitmap Graphic

Temporary bitmap graphics are bitmap files that are lost when the coder is powered off. They are standard 2 colour bitmap files that are stored in RAM and will be lost when the coder loses power. They are designed to be used in applications when a new image needs to be printed for each print, and host software will download a new bitmap to achieve this. It is the responsibility of the host system to ensure that these bitmaps do not use up all of the coders RAM (there is approximately 16MB of RAM storage available). The host system should either delete the bitmaps after use or use a repeating pattern of names (e.g. 0.bmp to 9.bmp) to ensure that bitmaps are overwritten. Temporary bitmaps use RAM that is used by the CLARiTY Update system. It may be necessary to power a coder off and on that has a large number of temporary bitmaps installed in order to perform a CLARiTY Update.

Sample Message

The following byte stream contains a File Request message for a file called Strawberry.xml of type Ciff File.

```
HOST> 0x03 0x50 0x1A 0x00 0x00 0x00 0x01 0x00 0x10 0x00 0x53 0x74 0x72 0x61
      0x77 0x62 0x65 0x72 0x72 0x79 0x2E 0x78 0x6D 0x6C 0x00 0x00
```

File Request Unicode (FRU – 0x5032)

This message is the same as the FRQ message except that the file name and the sub-directory name (if supplied) are specified using Unicode characters. This message is available in protocol version 4 or later.

The format of this message is as follows:

Name	Size	Description
Message ID	2 Bytes	0x5032
Message Length	4 Bytes	Variable length > 12
File Type	2 Bytes	A value that identifies the type of file that is requested.
File Name String Length	4 Bytes	The length of the file name string in bytes, including a NULL terminator.
File Name String	n Bytes	The name of the file requested as a Unicode string. This will not contain any path information but may contain an extension.
Additional Data	n Bytes	Contains any optional data for the given file type. The file types that support optional data are listed below. If no optional data is specified for the given file type, then no additional data section will be sent.

The additional data part of the message is the same as for the FRQ message with the exception of the Directory Listing request, which has the following format:

Name	Size	Description
Listing Content Flags	4 Bytes	Contains a number of bit flags determining the content of the listing see below
Sub-directory String Length	4 Bytes	The length of the sub-directory string, including a NULL terminator.
Sub-directory String	n Bytes	The name of the sub-directory to produce the listing of as a Unicode string.

Please refer to the documentation for the FRQ message for full details.

File Transmit (FTX – 0x5004)

This message transmits a file from either the Zipher printer to the Host or from the Host machine to the printer. The Host may send this message to the Zipher printer whenever it wishes to download a file. The printer will only use this message in response to a file request (FRQ) message.

In the scenario of the Zipher Printer receiving this message, the machine will respond with a NUL message if the transmission is successful and there is sufficient space to store the file. If there is insufficient space, a Protocol Error with the value insufficient disk space will be returned. If the file name or file type is invalid, the printer will return a Protocol Error with invalid file specification message.

Note that the Zipher Printer needs to maintain an amount of free disk space for its own internal use. This level is set at 50KB and the printer will return a Protocol Error with the value insufficient disk space if there will be less than 50KB of disk space left after the file download.

In the scenario of the Host receiving this message, no response for success or failure is required. If the host receives something that is invalid it should simply re-request the same file via the File Request (FRQ) command.

The format of this message is as follows:

Name	Size	Description
Message ID	2 Bytes	0x5004
Message Length	4 Bytes	Variable length > 14
File Type	2 Bytes	A value that identifies the type of file that is requested.
File Size	4 Bytes	The size of the file in bytes.
File Name String Length	2 Bytes	The length of the file name string, including a NULL terminator and any pad character to make the length even.
File Name String	n Bytes	The name of the file requested. This will not contain any path information but may contain an extension.
File Data	n Bytes	The binary data that composes the file. This will be even length padded; the file size indicates how much of this data is valid.

The supported file types are defined in the file request (FRQ) message.

Sample Message

The following byte stream contains a File Transmit of a message containing a file called Strawberry.xml of type Ciff File that is 7008 bytes long. The actual file data has been omitted for clarity.

```
HOST> 0x04 0x50 0x7E 0x1B 0x00 0x00 0x01 0x00 0x60 0x1B 0x00 0x00 0x10 0x00
      0x53 0x74 0x72 0x61 0x77 0x62 0x65 0x72 0x72 0x79 0x2E 0x78 0x6D 0x6C
      0x00 0x00 0x…
```

File Transmit Unicode (FTU – 0x5033)

This message is the same as the FTX message except that the file name is specified using Unicode characters. This message is available in protocol version 4 or later.

The format of this message is as follows:

Name	Size	Description
Message ID	2 Bytes	0x5033
Message Length	4 Bytes	Variable length > 16
File Type	2 Bytes	A value that identifies the type of file that is requested.
File Size	4 Bytes	The size of the file in bytes.
File Name String Length	4 Bytes	The length of the file name string in bytes, including a NULL terminator.
File Name String	n Bytes	The name of the file requested as a Unicode string. This will not contain any path information but may contain an extension.
File Data	n Bytes	The binary data that composes the file. This will be even length padded; the file size indicates how much of this data is valid.

Please refer to the documentation for the FTX message for details of this message.

File Transmit Unicode with Date (FTD – 0x503A)

This message is the same as the FTU message except that it allows the date the file was last modified to be specified (this can be used to verify the correct copy of a file is present in the printer using the directory list feature). It also allows the host to specify what happens when the file is received. This message is available in protocol version 8 or later.

The format of this message is as follows:

Name	Size	Description
Message ID	2 Bytes	0x5039
Message Length	4 Bytes	Variable length > 30
File Type	2 Bytes	A value that identifies the type of file that is requested.
File Size	4 Bytes	The size of the file in bytes.
File Transfer Flags	4 Bytes	Flags indicating what to do on receipt of the file.
Modify Year	2 Bytes	The year that the file was last modified (e.g. 2007).
Modify Month	1 Byte	The month that the file was last modified (e.g. 7).
Modify Day Of Week	1 Byte	The day of the week the file was last modified (Sunday = 0, Monday = 1, and so on).
Modify Day	1 Bytes	The day of the month the file was last modified.
Modify Hour	1 Bytes	The hour if the day that the file was last modified.
Modify Minute	1 Bytes	The minute of the hour that the file was last modified.
Modify Second	1 Bytes	The second of the minute that the file was last modified.
Modify Millisecond	2 Bytes	The millisecond of the second that the file was last modified.
File Name String Length	4 Bytes	The length of the file name string in bytes, including a NULL terminator.
File Name String	n Bytes	The name of the file requested as a Unicode string. This will not contain any path information, but may contain an extension.
File Data	n Bytes	The binary data that composes the file. This will be even length padded; the file size indicates how much of this data is valid.

The File Transfer Flags are defined as follows:

Name	Bit Number	Value	Description
FTDF_IGNOREDEPENDENCIES	0	0x00000001	Indicates that CLARiTY should not request any dependant files (e.g. Graphics and Fonts for a Job). This should be used when the host is managing file transfer dependencies.
FTDF_PRESERVELASTDOWNLOADED	1	0x00000002	Preserves the value of the last downloaded entry in the Job file. Only has an effect if the file type is CIFF File (Job File).
FTDF_REMOVEJOBPARAMS	2	0x00000004	Removes all job parameters from a job file when it is received. Only has an effect if the file type is CIFF File (Job File).

FTDF_RELATIVEPATH	3	0x00000008	Indicates that the file name contains path information relative to the root of the directory indicated by the file type (e.g. “150g/Tomato.ciff” would be stored in a sub directory called 150g under Jobs). When this flag is not set all sub directory information is discarded.

Please refer to the documentation for the FTU message for details of this message.

File Not Found (FNF - 0x5005)

This message is used to indicate that the requested file is no present or not valid. This message is used in response to any File Request message when the recipient cannot transmit the file.

In the scenario of the Zipher Printer receiving this message, the printer will respond with a NUL message to hand control back to the host, or with a file request (FRQ) to re-request the file.

The format of this message is as follows:

Name	Size	Description
Message ID	2 Bytes	0x5005
Message Length	4 Bytes	6 (or 10 if optional error code present)
Error Code	4 Bytes (optional)	Indicates the reason the file could not be found.

The defined Error Codes are as follows:

Name	Value	Description
EFNF_FILE_NOT_FOUND	0x0000	The file cound not be found at the requested location.
EFNF_COULD_NOT_LOAD_CIFF	0x0001	The CIFF file could not be loaded (corrupt).
EFNF_COULD_NOT_CONVERT_CIFF	0x0002	The CIFF file could not be converted for use with the printer (contains unsupported features).
EFNF_COULD_NOT_RESOLVE_DB	0x0003	Required database information for the Job could not be found in the database.

Sample Message

The following byte stream contains a file not found message.

```
HOST> 0x05 0x50 0x06 0x00 0x00 0x00
```

Poll for Requests (PFR – 0x5006)

This message is used by the host system to enquire whether the Zipher printer has any requests. The Host should send the message to the Zipher printer at regular intervals. The frequency of the polling is dictates the responsiveness of the system, and should not be more frequent that 10 per second.

The message is a Host to Zipher printer only message. The valid responses from the printer are NUL if no requests are pending, FRQ if the printer requires a file and FTX if the printer wishes to transmit a file. If protocol version 4 or later is in force, FRU and FTU messages will be sent instead of FRQ and FTX respectively.

The need for polling the printer can be eliminated by setting up the advise mechanism. See the documentation for the Advise of Information Update (AIU) command.

The format of this message is as follows:

Name	Size	Description
Message ID	2 Bytes	0x5006
Message Length	4 Bytes	6

This response has no message body.

Sample Message

The following byte stream contains a poll for requests message.

```
HOST> 0x06 0x50 0x06 0x00 0x00 0x00
```

Select Job (SEL – 0x5007)

This message allows the host system to select a new job on the Zipher printer. It is a Host to Zipher printer only message.

The responses to this message depend upon the state of the printer. If the job is present in the printers' database, and the printer is able to select it, a NUL (or JUC – see below) message will be returned. If the job is not present in the database of the host, then a FRQ message will be sent back. If the job has a print limit, and the print limit is locked into the machine, then a ERR message will be returned with the Job Select Fail – previous selection is locked in response. Other types of ERR message may be returned depending upon the type of failure, see the protocol error documentation for more information.

The format of this message is as follows:

Name	Size	Description
Message ID	2 Bytes	0x5007
Message Length	4 Bytes	Variable length > 18
File Name String Length	2 Bytes	The length of the file name string, including a NULL terminator and any pad character to make the length even.
File Name String	n Bytes	The name of the file requested. This should not contain any path information, but may contain an extension.
Job Flags	4 Bytes	Flags indicating the behaviour of the job
Product Limit/Line ID	4 Bytes	A limit on the number of products that this job may be printed on. Only contains a valid number if the JF_PRODLIMIT/JF_LINEUPDATE bit is set in the Job Flags.
Ciff Data File Length	2 Bytes	The length of the Ciff Data file string, including a NULL terminator and any pad character to make the length even.
Ciff Data File String	n Bytes	The Ciff data file.

The flags defined for the Job Flags are:

Name	Bit Number	Value	Description
JF_PRODLIMIT	0	0x00000001	This Job has a limited number of products it can print on.
JF_PRODLMTLOCK	1	0x00000002	The product limit is locked into the machine and cannot be overridden by a product selection/power off
JF_USECURRENTDATA	2	0x00000004	Instructs the selection software **NOT** to follow the rules within the CIFF, but to use the current data for all variable information.
JF_NOBATCHCOUNTRESET	3	0x00000008	Instructs the Printer not to reset its batch count as a result of this selection.
JF_CLEARLASTPRODLIMIT	4	0x00000010	Instructs the Printer to clear any existing product limit, and forces the new job to be selected immediately
JF_BINARYCIFFDATA	5	0x00000020	Indicates that the binary CIFF data file format should be used instead of the default.

JF_RETURNUPDATEID	6	0x00000040	(Protocol revision 7 or later) Indicates that the id of the update as it is added to the job update queue should be returned to the host. If this flag is included, the printer will confirm successful receipt of the message with JUC instead of NUL.
JF_LINEUPDATE	7	0x00000080	(Protocol revision 8 or later) Indicates that the Product Limit should be treated as a line number.
JF_EXPIREDPRODLIMIT	8	0x00000100	(Protocol revision 9 or later) Indicates that the Product Limit should be treated as expired/complete.
JF_RESETIMAGESEQUENCE	9	0x00000200	(Protocol revision 9 or later) Ignored for SEL commands, used by JDU command.
JF_AUTOCLEARLIMITFAULT	10	0x00000400	(Protocol revision 10 or later) Automatically clear the "(E1005) Print Limit Exceeded" fault and go online if this fault was raised and it was the only fault raised.
JF_TEMPORARYCIFF	11	0x00000800	(Protocol revision 11 or later) Indicates the Job to be selected is a Temporary CIFF stored in RAM. Job will be lost on power off.

The product limit allows the Host to ensure that only a limited number of products get printed with this job information. The value of the Product Limit is only valid if the JF_PRODLIMIT is set. When the product limit is set on its own, the printer will print the number of products specified and the stop printing until a new selection or job data update is received. However, the limit may be overridden by selecting a new job, sending a job data update, or by switching the machine off. To lock the product limit into the machine and ensure that the specified number of products is printed before any further selection is allowed set the JF_PRODLMTLOCK bit. There are only 3 ways of removing a locked product limit from the machine, to complete the run of products, or to send the CMD message with the remove product limit command, or to send a new SEL message with the JF_CLEARLASTPRODLIMIT flag set.

The JF_CLEARLASTPRODLIMIT flag forces the printer to make the new Job the active image immediately, regardless of the current state of the Job.

The JF_LINEUPDATE flag changes the operation of the SEL message from a Job selection to a Line Assignment message. This assigns the job to the given line, where the Product Limit field indicates the line number. In order for this to be message to accepted the printer must be in line selection mode otherwise an ERR(RFF_NOTINLINESELECTMODE) will be returned. An ERR(RFF_INVALIDLINEID) will be returned if the line id is invalid. Valid line numbers range from 1 to the maximum number of lines setup at the printer. There are two special job names supported in line selection mode, an empty job name indicates that the printer should fault out if product is received from that line, and a job name of * indicates that the printer should allow product to pass unprinted from a given line.

The normal mode of operation is to select the Ciff file with its default data, this is achieved by setting the Ciff Data File Length to 0 and omitting the Ciff Data File string. If however, the Host wishes to override the default data with its own data, the Ciff Data File Length should be set to the length of the provided Ciff data file. If the JF_BINARYCIFFDATA flag is set the binary format must be used, otherwise the XML format must be used. Reference the Ciff Data File Design Document for a description of the format of the string.

The JF_EXPIREDPRODLIMIT flag should be used in random imaging applications where the data for each print is provided by the Job Data Update (JDU) command. In these applications it is often necessary to select a new job at the start of the run, but the job won't contain valid print data, and so must not be printed. To select a job and prevent printing the SEL message should set the Product Limit to a non-zero value (e.g. 1), and the flags should have at least the JF_PRODLIMIT and JF_EXPIREDPRODLIMIT flags set.

Sample Message

The following byte stream contains a Select message for a Job file called Strawberry.xml in specifies a locked product limit of 1 product and accepts the default product data.

```
HOST> 0x07 0x50 0x22 0x00 0x00 0x00 0x10 0x00 0x53 0x74 0x72 0x61 0x77 0x62
      0x65 0x72 0x72 0x79 0x2E 0x78 0x6D 0x6C 0x00 0x00 0x03 0x00 0x00 0x00
      0x01 0x00 0x00 0x00 0x00 0x00
```

Select Job Unicode (SLU – 0x5030)

This message is the same as SEL except that the job name is specified using Unicode characters. This message is available in protocol version 4 or later.

The format of this message is as follows:

Name	Size	Description
Message ID	2 Bytes	0x5030
Message Length	4 Bytes	Variable length > 18
File Name String Length	4 Bytes	The length of the file name string in bytes, including a NULL terminator.
File Name String	n Bytes	The name of the file requested as a Unicode string. This should not contain any path information, but may contain an extension.
Job Flags	4 Bytes	Flags indicating the behaviour of the job
Product Limit	4 Bytes	A limit on the number of products that this job may be printed on. Only contains a valid number if the JF_PRODLIMIT bit is set in the Job Flags.
Ciff Data File Length	4 Bytes	The length of the Ciff Data file string, including a NULL terminator and any pad character to make the length even.
Ciff Data File String	n Bytes	The Ciff data file.

Please refer to the documentation for the SEL command for details of the message.

Job Data Update (JDU - 0x5008)

This message allows the host system to update the data of the currently running job on the Zipher printer. It is a Host to Zipher printer only message.

The responses to this message depend upon the state of the printer. If the job data update works successfully, a NUL (or JUC – see below) message will be returned. If the job has a print limit, and the print limit is locked into the machine, then a ERR message will be returned with the Job Data Update Fail – previous selection is locked in response. Other types of ERR message may be returned depending upon the type of failure, see the protocol error documentation for more information.

The format of this message is as follows:

Name	Size	Description
Message ID	2 Bytes	0x5008
Message Length	4 Bytes	Variable length > 16
Job Flags	4 Bytes	Flags indicating the behaviour of the job
Product Limit	4 Bytes	A limit on the number of products that this job may be printed on. Only contains a valid number if the JF_PRODLIMIT/JF_LINEUPDATE bit is set in the Job Flags.
Ciff Data File Length	2 Bytes	The length of the Ciff Data file string, including a NULL terminator and any pad character to make the length even.
Ciff Data File String	n Bytes	The Ciff data file.

The flags defined for the Job Flags are:

Name	Bit Number	Value	Description
JF_PRODLIMIT	0	0x00000001	This Job has a limited number of products it can print on.
JF_PRODLMTLOCK	1	0x00000002	The product limit is locked into the machine and cannot be overridden by a product selection/power off
JF_CLEARLASTPRODLIMIT	4	0x00000010	Instructs the Printer to clear any existing product limit, and forces the new job to be selected immediately
JF_BINARYCIFFDATA	5	0x00000020	Indicates that the binary CIFF data file format should be used instead of the default.
JF_RETURNUPDATEID	6	0x00000040	(Protocol revision 7 or later) Indicates that the id of the update as it is added to the job update queue should be returned to the host. If this flag is included, the printer will confirm successful receipt of the message with JUC instead of NUL.
JF_LINEUPDATE	7	0x00000080	(Protocol revision 8 or later) Indicates that the Product Limit should be treated as a line number.
JF_EXPIREDPRODLIMIT	8	0x00000100	(Protocol revision 9 or later) Indicates that the Product Limit should be treated as expired/complete.
JF_RESETIMAGESEQUENCE	9	0x00000200	(Protocol revision 9 or later) Indicates that the image sequence should be reset.

JF_AUTOCLEARLIMITFAULT	10	0x00000400	(Protocol revision 10 or later) Automatically clear the "(E1005) Print Limit Exceeded" fault and go online if this fault was raised and it was the only fault raised.
JF_TEMPORARYCIFF	11	0x00000800	(Protocol revision 11 or later) Has no effect on Job Updates and will be ignored.

See the Select Job documentation for more information on the usage and behaviour of product limits. Reference the Ciff Data File Design Document for a description of the format of the string.

The JF_LINEUPDATE flag changes the operation of the JDU message from a Job Data Update to a Line Data Update message. This updates the printing information on the given line, where the Product Limit field indicates the line number. In order for this to be message to accepted the printer must be in line selection mode otherwise an ERR(RFF_NOTINLINESELECTMODE) will be returned. An ERR(RFF_INVALIDLINEID) will be returned if the line id is invalid. Valid line numbers range from 1 to the maximum number of lines setup at the printer.

Sample Message

The following byte stream contains a Job Data Update message; it specifies a locked product limit of 1 product and supplies 166 bytes of ciff data file.

```
HOST> 0x08 0x50 0xB6 0x00 0x00 0x00 0x03 0x00 0x00 0x00 0x01 0x00 0x00 0x00
      0xA6 0x00 0x…
```

Job Update Confirmation (JUC – 0x5022)

This message confirms receipt of a job selection (SEL/SLU) or job data update (JDU) message, and includes the update ID. It is a Zipher printer to Host only message. Supported in protocol revision 7 or later.

The format of this message is as follows:

Name	Size	Description
Message ID	2 Bytes	0x5022
Message Length	4 Bytes	10
Update ID	4 Bytes	The ID of the job update. This is used in other messages such as the asynchronous notifications of the job rendering and selection process.

Multiple Job Data Update (MJU – 0x503D)

This message allows the host system to update the data of the currently running job on the Zipher printer with multiple binary ciff data. It is a Host to Zipher printer only message.

The responses to this message depend upon the state of the printer. If the multiple job data update works successfully, a NUL (or JUC) message will be returned. If the job has a print limit, and the print limit is locked into the machine, then a ERR message will be returned with the Job Data Update Fail – previous selection is locked in response. If the number of job updates specified is greater than the number of vacancies left in the queue then a - RFF_IMAGEUPDATEQUEUEFULL message will be returned. Other types of ERR message may be returned depending upon the type of failure, see the protocol error documentation for more information.

The format of this message is as follows:

Name	Size	Description
Message ID	2 Bytes	0x500C
Message Length	4 Bytes	Variable length > 18
Number of Updates	2 Bytes	The number of job updates which follow (m)
Job Flags 1	4 Bytes	Flags indicating the behaviour of the job
Product Limit 1	4 Bytes	A limit on the number of products that this job may be printed on. Only contains a valid number if the JF_PRODLIMIT/JF_LINEUPDATE bit is set in the Job Flags.
Ciff Data File Length 1	2 Bytes	The length of the Ciff Data file string, including a NULL terminator and any pad character to make the length even(n).
Ciff Data File String 1	n Bytes	The first Ciff data file.
	.	
	.	
	.	
Job Flags m	4 Bytes	Flags indicating the behaviour of the job
Product Limit m	4 Bytes	A limit on the number of products that this job may be printed on. Only contains a valid number if the JF_PRODLIMIT/JF_LINEUPDATE bit is set in the Job Flags.
Ciff Data File Length m	2 Bytes	The length of the Ciff Data file string, including a NULL terminator and any pad character to make the length even(n).
Ciff Data File String m	n Bytes	The mth Ciff data file.
	.	

The flags defined for the Job Flags are:

Name	Bit Number	Value	Description
JF_PRODLIMIT	0	0x00000001	This Job has a limited number of products it can print on.
JF_PRODLMTLOCK	1	0x00000002	The product limit is locked into the machine and cannot be overridden by a product selection/power off
JF_CLEARLASTPRODLIMIT	4	0x00000010	Instructs the Printer to clear any existing product limit, and forces the new job to be selected immediately
JF_BINARYCIFFDATA	5	0x00000020	Must be present. Indicates that the binary CIFF data file form is used. The XML format is not implemented for the MJU message.

JF_RETURNUPDATEID	6	0x00000040	(Protocol revision 7 or later) Indicates that the id of the last update in the MJU as it is added to the job update queue should be returned to the host. If this flag is included, the printer will confirm successful receipt of the message with JUC instead of NUL.
JF_LINEUPDATE	7	0x00000080	(Protocol revision 8 or later) Indicates that the Product Limit should be treated as a line number.
JF_EXPIREDPRODLIMIT	8	0x00000100	(Protocol revision 9 or later) Indicates that the Product Limit should be treated as expired/complete.
JF_RESETIMAGESEQUENCE	9	0x00000200	(Protocol revision 9 or later) Indicates that the image sequence should be reset.
JF_AUTOCLEARLIMITFAULT	10	0x00000400	(Protocol revision 10 or later) Automatically clear the "(E1005) Print Limit Exceeded" fault and go online if this fault was raised and it was the only fault raised.

See the Select Job documentation for more information on the usage and behaviour of product limits. Reference the Ciff Data File Design Document for a description of the format of the string.

The JF_LINEUPDATE flag changes the operation of the MJU message from a Job Data Update to a Multiple Line Data Update message. This updates the printing information on the given line, where the Product Limit field indicates the line number. In order for this message to be accepted the printer must be in line selection mode otherwise an ERR(RFF_NOTINLINESELECTMODE) will be returned. An ERR(RFF_INVALIDLINEID) will be returned if the line id is invalid. Valid line numbers range from 1 to the maximum number of lines setup at the printer.

Sample Message

The following byte stream contains a Multiple Job Data Update message; it specifies 2 binary ciff files with a product limit of 1 in the first instance.

```
HOST> 0x3d 0x50    0x3a 0x01 0x00 0x00   0x02 0x00   0x61 0x00 0x00 0x00
      0x01 0x00 0x00 0x00    0x8c 0x00
```

Advise on Information Update (AIU – 0x5009)

This message allows the Host to configure the Zipher printer to send out a message when certain key events occur. The message is Host to Zipher printer only.

Using the Advise message allows the host to avoid the overhead of polling the machine on a regular basis, by instructing the machine to send out a message when an event occurs. In order for this to work reliably the host has to be capable of full duplex communications. That is, the host must be capable of transmitting and receiving simultaneously as the host could be in the process of sending a message to the printer when the event occurs, and the Zipher printer starts transmitting its message.

For more information on the pitfalls of using the AIU message consult the "Using the AIU Message" section at the back of this document.

The format of the Advise message is as follows:

Name	Size	Description
Message ID	2 Bytes	0x5009
Message Length	4 Bytes	10
Advise Flags	4 Bytes	A set of flags indicating what events the Host wishes to be informed of

The events that the printer can advise upon are:

Name	Bit Number	Value	Description
AF_ERROR	1	0x00000002	There has been a change in the printers' error status – a new error has occurred, or an error has been cleared. When this event occurs the Zipher printer will send an IRP message containing the state and status tag.
AF_WARNING	2	0x00000004	There has been a change in the printers' warning status – a new warning has occurred, or a warning has been cleared. When this event occurs the Zipher printer will send an IRP message containing the state and status tag.
AF_ONLINE	3	0x00000008	There has been a change in the printers' online/offline status. When this event occurs the Zipher printer will send an IRP message containing the state and status tag.
AF_JOBSELECT	4	0x00000010	There has been a new job selected (not by binary communications). When this event occurs the Zipher printer will send an IRP message containing the Job Information tag (JIU for protocol version 4 or later; JIN otherwise).

All other bits within the Advise flags **must** be set to 0 as additional flags may be added in the future, which may cause unexpected messages to be sent.

Sample Message

The following byte stream contains an Advise message requesting to be informed when a product is printed, and the printer goes on and offline. It also requests a timed update.

```
HOST> 0x09 0x50 0x0A 0x00 0x00 0x00 0x89 0x00 0x00 0x00
```

Asynchronous Information Subscription (AIS – 0x5035)

This message allows the Host to configure the Zipher printer to send out a message when certain key events occur. The message is Host to Zipher printer only. Asynchronous Information Subscription is available from protocol version 7 onwards and supersedes the AIU message which should no longer be used.

Using the subscription message allows the host to avoid the overhead of polling the machine on a regular basis, by instructing the machine to send out a message when an event occurs. In order for this to work reliably the host has to be capable of full duplex communications. That is, the host must be capable of transmitting and receiving simultaneously as the host could be in the process of sending a message to the printer when the event occurs, and the Zipher printer starts transmitting its message.

For more information on the pitfalls of using the AIS message consult the "Using the AIS Message" section at the back of this document.

The format of the subscription message is as follows:

Name	Size	Description
Message ID	2 Bytes	0x5025
Message Length	4 Bytes	Variable length > 11
Operation Code	1 Byte	Determines whether subscription is set, added to or deleted
Item Count	4 Bytes	The number of subscription items to follow
Subscription Id	2 Bytes	Identifies a subscription to be altered
Flags	4 Bytes	Flags to be set for the subscription item
`		
`		

The Operation Codes for altering subscriptions are:

Name	Value	Description
AIS_ADD	0x00	Add the subscriptions in this message to any existing ones.
AIS_DEL	0x01	Delete the subscriptions in this message leaving any others active.
AIS_SET	0x02	Set the active subscriptions to be the ones in this message deleting any others that are active.

The Subscription Identifiers a host can send are:

Name	Value	Description
AIS_JOB_SELECT	0x0000	Notify when a Job selection has occurred.
AIS_TRANSMITFIELD	0x0001	Notify when a field marked as a Transmitted Field is loaded as part of a Job selection.
AIS_PRINTER_IS_ONLINE	0x0002	Notify when the printer goes online.
AIS_PRINTER_ENTERS_WARNING	0x0003	Notify when the printer goes into a warning state.
AIS_PRINTER_IS_BUSY	0x0004	Notify when the printer is mechanically active.
AIS_PRINTER_STARTS_PRINTING	0x0005	Notify when the printer starts printing.
AIS_LABELLER_MAY_FEED	0x0006	Notify when an external labeller is allowed to feed.
AIS_PRINT_CYCLE_STARTS	0x0007	Notify when a print cycle starts.
AIS_PRINTER_IS_OFFLINE	0x0008	Notify when the printer goes offline.
AIS_PRINTER_LEAVES_WARNING	0x0009	Notify when the printer is no longer in a warning state.

AIS_PRINTER_IS_NOT_BUSY	0x000A	Notify when the printer is mechanically inactive.
AIS_PRINTER_FINISHES_PRINTING	0x000B	Notify when the printer finished printing.
AIS_LABELLER_MAY_NOT_FEED	0x000C	Notify when an external labeller is not allowed to feed.
AIS_PRINT_CYCLE_COMPLETES	0x000D	Notify when a print cycle completes.
AIS_JOB_UPDATE_Q_IS_FULL	0x000E	Notify when the printer job update queue becomes full.
AIS_JOB_UPDATE_Q_IS_NOT_FULL	0x000F	Notify when the printer job update queue is no longer full.
AIS_NEW_ALLOCATION_RECEIVED	0x0010	Notify when the printer has received a new allocation.
AIS_JOB_ALLOCATION_COMPLETE	0x0011	Notify when the printer has completed printing of an allocation.
AIS_PRINT_FAILED	0x0012	Notify when a print fails.
AIS_PRINTER_LEAVES_FAULT	0x0013	Notify when the printer is no longer in a fault state.
AIS_PRINTER_ENTERS_FAULT	0x0014	Notify when the printer goes into a fault state.
AIS_PRINT_SIGNAL_IGNORED	0x0015	Notify when an external print signal is not acted on.
AIS_RENDERING_COMPLETE	0x0016	Notify when the image for a new or update job has been successfully rendered.
AIS_RENDERING_FAILED	0x0017	Notify when the image for a new or updated job failed to render.
AIS_JOB_UPDATE_Q_IS_EMPTY	0x0018	Notify when the printer job update queue becomes empty
AIS_JOB_UPDATE_Q_IS_NOT_EMPTY	0x0019	Notify when the printer job update queue is no longer empty
AIS_JOB_ALLOCATION_ABORTED	0x001A	Notify when the printer aborts an allocation without completing it, e.g. when the job update queue is cleared.
AIS_JOB_UPDATE_Q_IS_LOW	0x001B	Notify when the printer job update queue falls below the LOW_Q_LEVEL threshold
AIS_JOB_UPDATE_Q_IS_HIGH	0x001C	Notify when the printer job update queue rises above the HIGH_Q_LEVEL threshold
AIS_IASSURE_CAPTURE_READY	0x001D	Notify when iAssure has recorded a new capture, and at least one second has passed since the previous iAssure capture, or no previous capture exists.

The Flags associated with subscriptions are:

Name	Value	Description
LOW_PRIORITY	0x00000000	Notification of the event subscribed to will be sent out when a block of notifications have built up, when the next high priority event occurs or after approx.. 1/3 second whichever is the sooner.
HIGH_PRIORITY	0x00000080	Notification of the event will be sent out immediately on its occurrence. Other low priority notifications which have built up will also be flushed.

Note: Flags values are ignored if the Operation is AIS_DEL. The HIGH_PRIORITY flag should only be used where performance is critical and for 1 subscription item. Printer performance may be adversely affected is multiple HIGH_PRIORITY subscriptions are made.

Sample Message

The following byte stream contains a Subscription message requesting to be notified only (AIS_SET) when the printer goes on and offline. Online being low priority and offline high.

```
HOST> 0x35 0x50 0x17 0x00 0x00 0x00 0x02 0x02 0x00 0x00 0x00 0x02 0x00 0x00
      0x00 0x00 0x00 0x08 0x00 0x80 0x00 0x00 0x00

This breaks down as:
      Message ID              0x35 0x50
      Message Length          0x17 0x00 0x00 0x00
      AIS_SET                 0x02
      Item Count              0x02 0x00 0x00 0x00
      AIS_PRINTER_IS_ONLINE   0x02 0x00
      LOW_PRIORITY            0x00 0x00 0x00 0x00
      AIS_PRINTER_IS_OFFLINE  0x08 0x00
      HIGH_PRIORITY           0x80 0x00 0x00 0x00
```

Asynchronous Information Response (AIR – 0x5036)

This message provides information about the status of the Zipher Printer in response to an Asynchronous Information Request. The information that is returned is dependent upon the current set of active subscriptions and the events that have occurred on the printer to cause the message to be sent. The message uses a tag-based format to provide multiple blocks of status information.

The only valid use of this message is as a response from the Zipher Printer to the Host machine. It is only returned indirectly as a response to an Asynchronous Information Request (AIS - 0x5025). See the Asynchronous Information Subscription message for more details.

The format of this message is as follows:

Name	Size	Description
Message ID	2 Bytes	0x5036
Message Length	4 Bytes	Variable length > 10
Number of Tags	4 Bytes	Number of Information Tags
Information Tag ID	2 Bytes	Information Tag IDs of the information the host is interested in.
Information Tag Size	4 Bytes	Size of the data for this tag including ID and length
Information Tag Data	n Bytes	Tag dependant data, size given by the Tag Size – 4, structure of the data is given under each information tag description.

For each tag in the number of tags section there will be an Information Tag ID, and Information Tag Size and a block of tag data. The next tag information will immediately follow the end of the previous tag block.

The Information Tag IDs are identical to the Subscription IDs sent with the Subscription message.

The following sections describe the content and format of each information tag.

Job Select (AIS_JOBSELECT – 0x0000)

The Job Select Tag indicates that a Job Selection has taken place, either by direct user interaction with the printer or on receipt of a Job Select message from a connected device.

The format of the Job Select Information tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x0000
Information Tag Size	4 Bytes	30 + length of Job Name string
Length of Job Name string	4 Bytes	The length of the string to follow
Job Name string	N Bytes	The name of the Job that has been selected
Job Flags	4 Bytes	For an explanation of Job Flags see Select Job (SEL – 0x5007) on Page 42
Products Remaining	4 Bytes	The number of prints remaining in the current allocation.
Hours	2 Bytes	The Hours component of the selection Date and Time.
Minutes	2 Bytes	The Minutes component of the selection Date and Time.
Seconds	2 Bytes	The Seconds component of the selection Date and Time.
Day	2 Bytes	The Day component of the selection Date and Time.

Month	2 Bytes	The Month component of the selection Date and Time.
Year	2 Bytes	The Year component of the selection Date and Time.

Transmitted Field (AIS_TRANSMITFIELD – 0x0001)

The Transmitted Field Tag carries the data from a field in the Job being selected or printed which is marked for transmission.

The format of the Transmitted Field Information tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x0001
Information Tag Size	4 Bytes	10 + length of Field Data string
Length of Field Data string	4 Bytes	The length of the string to follow
Field Data string	n Bytes	The Field Data for the transmitted field

Printer Is Online (AIS_PRINTER_IS_ONLINE – 0x0002)

The Printer Is Online Tag indicates that the printer is online.

The format of the Printer Is Online Information tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x0002
Information Tag Size	4 Bytes	10
State	4 Bytes	0x00000001

Printer Enters Warning (AIS_PRINTER_ENTERS_WARNING – 0x0003)

The Printer Enters Warning Tag indicates that the printer has at least one active warning.

The format of the Printer Enters Warning Information Tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x0003
Information Tag Size	4 Bytes	10
State	4 Bytes	0x00000001

Printer Is Busy (AIS_PRINTER_IS_BUSY- 0x0004)

The Printer Is Busy Tag indicates that the printer has entered the mechanically active part of the print cycle.

The format of the Printer Is Busy Information Tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x0004
Information Tag Size	4 Bytes	10
State	4 Bytes	0x00000001

Printer Starts Printing (AIS_PRINTER_STARTS_PRINTING – 0x0005)

The Printer Starts Printing Tag indicates that the printer has entered the print application part of the print cycle.

The format of the Printer Starts Printing Information Tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x0005
Information Tag Size	4 Bytes	10
State	4 Bytes	0x00000001

Labeller May Feed (AIS_LABELLER_MAY_FEED – 0x0006)

The Labeller May Feed Tag indicates that an external labeller may safely perform a feed operation at this point.

The format of the Labeller May Feed Information Tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x0006
Information Tag Size	4 Bytes	10
State	4 Bytes	0x00000001

Print Cycle Starts (AIS_PRINT_CYCLE_STARTS – 0x0007)

The Print Cycle Starts Tag indicates that the printer has entered the print cycle.

The format of the Print Cycle Starts Information Tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x0007
Information Tag Size	4 Bytes	22
State	4 Bytes	0x00000001
Update ID	4 Bytes	The ID of the Job Update Queue item being processed
Print Number	4 Bytes	The index of this print cycle
Allocation	4 Bytes	The Allocation limit for the selected job

Printer Is Offline (AIS_PRINTER_IS_OFFLINE – 0x0008)

The Printer Is Offline Tag indicates that the printer is offline.

The format of the Printer Is Offline Information Tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x0008
Information Tag Size	4 Bytes	10
State	4 Bytes	0x00000000

Printer Leaves Warning (AIS_PRINTER_LEAVES_WARNING – 0x0009)

The Printer Leaves Warning Tag indicates that there are no active warnings on the printer.

The format of the Printer Leaves Warning Information Tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x0009
Information Tag Size	4 Bytes	10
State	4 Bytes	0x00000000

Printer Is Not Busy (AIS_PRINTER_IS_NOT_BUSY – 0x000A)

The Printer Is Not Busy Tag indicates that the printer has completed the mechanical part of the print cycle.

The format of the Printer Is Not Busy Information Tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x000A
Information Tag Size	4 Bytes	10
State	4 Bytes	0x00000000

Printer Finishes Printing (AIS_PRINTER_FINISHES_PRINTING – 0x000B)

The Printer Finishes Printing Tag indicates that the printer has completed the print application part of the print cycle.

The format of the Printer Finishes Printing Information Tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x000B
Information Tag Size	4 Bytes	10
State	4 Bytes	0x00000000

Labeller May Not Feed (AIS_LABELLER_MAY_NOT_FEED – 0x000C)

The Labeller May Not Feed Tag indicates that an external labeller may no longer safely perform a feed operation.

The format of the Labeller May Not Feed Information Tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x000C
Information Tag Size	4 Bytes	10
State	4 Bytes	0x00000000

Print Cycle Completes (AIS_PRINT_CYCLE_COMPLETES – 0x000D)

The Print Cycle Completes Tag indicates that the printer has completed a print cycle.

The format of the Print Cycle Completes Information Tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x000D
Information Tag Size	4 Bytes	10
State	4 Bytes	0x00000000

Job Update Queue Is Full (AIS_JOB_UPDATE_Q_IS_FULL – 0x000E)

The Job Update Queue Is Full Tag indicates that no more job updates can be accepted by the printer because the configured maximum Job Update Queue length has been reached.

The format of the Job Update Queue Is Full Information Tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x000E
Information Tag Size	4 Bytes	10
State	4 Bytes	0x00000001

Job Update Queue Is Not Full (AIS_JOB_UPDATE_Q_IS_NOT_FULL – 0x000F)

The Job Update Queue Is Not Full Tag indicates that one or more slots are now available in the printer's Job Update Queue.

The format of the Job Update Queue Is Not Full Information Tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x000F
Information Tag Size	4 Bytes	10
State	4 Bytes	0x00000000

New Allocation Received (AIS_NEW_ALLOCATION_RECEIVED – 0x0010)

The New Allocation Received Tag indicates that the printer has received a new print allocation as a result of a Job Data Update.

The format of the New Allocation Received Information Tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x0010
Information Tag Size	4 Bytes	14
Update ID	4 Bytes	The ID of the Job Update Queue item being processed
Allocation	4 Bytes	The new allocation

Job Allocation Complete (AIS_JOB_ALLOCATION_COMPLETE – 0x0011)

The Job Allocation Complete Tag indicates that allocation limit for the currently selected job has been reached. Printing will stop unless further job update queue entries exist.

The format of the Job Allocation Complete Information Tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x0011
Information Tag Size	4 Bytes	10
Update ID	4 Bytes	The ID of the Job Update Queue item completed

Print Failed (AIS_PRINT_FAILED – 0x0012)

The Print Failed Tag indicates that a print cycle was aborted due to an error.

The format of the Print Failed Information Tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x0012
Information Tag Size	4 Bytes	10
Update ID	4 Bytes	The ID of the Job Update Queue item for which a print failed

Printer Leaves Fault (AIS_PRINTER_LEAVES_FAULT – 0x0013)

The Printer Leaves Fault Tag indicates that there are no active faults on the printer.

The format of the Printer Leaves Fault Information Tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x0013
Information Tag Size	4 Bytes	10
State	4 Bytes	0x00000001

Printer Enter Fault (AIS_PRINTER_ENTERS_FAULT – 0x0014)

The Printer Enters Fault Tag indicates that the printer has one or more active faults.

The format of the Printer Enters Fault Information Tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x0014
Information Tag Size	4 Bytes	10
State	4 Bytes	0x00000000

Print Signal Ignored (AIS_PRINT_SIGNAL_IGNORED – 0x0015)

The Print Signal Ignored Tag indicates that one or more hardware print signals were not acted on.

The format of the Print Signal Ignored Information Tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x0015
Information Tag Size	4 Bytes	10
Count	4 Bytes	The Count of Print Signals ignored to date

Rendering Complete (AIS_RENDERING_COMPLETE – 0x0016)

The Rendering Complete Tag indicates that the new print image resulting from a Job Selection or Job Data Update has been successfully rendered.

The format of the Rendering Complete Information Tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x0016
Information Tag Size	4 Bytes	10
Update ID	4 Bytes	The ID of the Job Update Queue item rendered

Rendering Failed (AIS_RENDERING_FAILED – 0x0017)

The Rendering Failed Tag indicates that the new print image resulting from a Job Selection or Job Data Update has failed to render.

The format of the Rendering Failed Information Tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x0017
Information Tag Size	4 Bytes	10
Update ID	4 Bytes	The ID of the Job Update Queue item that failed to render

Job Update Queue Is Empty (AIS_JOB_UPDATE_Q_IS_EMPTY – 0x0018)

The Job Update Queue Is Empty Tag indicates that there are no more updates in the Job Update Queue.

The format of the Job Update Queue Is Empty Information Tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x0017
Information Tag Size	4 Bytes	10
State	4 Bytes	0x00000001

Job Update Queue Is Not Empty (AIS_JOB_UPDATE_Q_IS_NOT_EMPTY – 0x0019)

The Job Update Queue Is Not Empty Tag indicates that the printer's Job Update Queue contains at least one item.

The format of the Job Update Queue Is Not Empty Information Tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x0019
Information Tag Size	4 Bytes	10
State	4 Bytes	0x00000000

Job Allocation Aborted (AIS_JOB_ALLOCATION_ABORTED – 0x001A)

The Job Allocation Aborted Tag indicates that allocation for the currently selected job has been terminated before completion.

The format of the Job Allocation Aborted Information Tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x001A
Information Tag Size	4 Bytes	10
Update ID	4 Bytes	The ID of the Job Update Queue item aborted

Job Update Queue Is Low (AIS_JOB_UPDATE_Q_IS_LOW – 0x001B)

The Job Update Queue Is Low Tag indicates that the printer's Job Update Queue has fallen below the LOW_Q_LEVEL config. Parameter threshold

The format of the Job Update Queue Is Low Tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x001B
Information Tag Size	4 Bytes	10
State	4 Bytes	0x00000000

Job Update Queue Is High (AIS_JOB_UPDATE_Q_IS_HIGH – 0x001C)

The Job Update Queue Is High Tag indicates that the printer's Job Update Queue has risen above the HIGH_Q_LEVEL config. Parameter threshold

The format of the Job Update Queue Is Low Tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x001C
Information Tag Size	4 Bytes	10
State	4 Bytes	0x00000001

iAssure Capture Ready (AIS_IASSURE_CAPTURE_READY – 0x001D)

The iAssure Capture Ready tag indicates that iAssure has recorded a new capture and either one second has passed since the last iAssure capture was taken or no previous capture exists.

The format of the iAssure Capture Ready tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x001D
Information Tag Size	4 Bytes	10
State	4 Bytes	0x00000001

Command (CMD – 0x500A)

This message is used by the Host to command the Zipher Printer to perform an operation. The message is Host to Zipher Printer only.

The printer will respond with a NUL command when it completes the requested operation. If the command is invalid a Protocol Error of Invalid command will be returned. Other protocol errors may be returned depending upon the command issued. See the protocol error response for more information.

The format of this message is as follows:

Name	Size	Description
Message ID	2 Bytes	0x500A
Message Length	4 Bytes	8
Command ID	2 Bytes	ID of the command to perform

The defined Command IDs are:

Name	Value	Description
CMD_PRINT	0x0001	Causes the printer to perform a print
CMD_START	0x0003	Puts the machine online. This command is only valid if the machine is in an error free offline state.
CMD_STOP	0x0004	Puts the machine offline. This command is only valid if the machine is online
CMD_STARTUP	0x0005	Starts the machine up and sends it automatically online. This command is only valid if the machine is shutdown.
CMD_SHUTDOWN	0x0006	Shuts the machine down. This command is only valid if the machine is not shutdown.
CMD_SNAPSHOT	0x0007	Creates a Bitmap image of the current CLARiTY screen and adds it to the transmit queue so that it will be returned on the next PFR message
CMD_CLEARQUEUE	0x0008	Erases all entries in the job update queue.
CMD_PALLETPRINTAPPLY1	0x0091	Causes the printer to perform print and apply sequence 1 (this the same as CMD_PRINT)
CMD_PALLETPRINTAPPLY2	0x0092	Causes the printer to perform print and apply sequence 2
CMD_PALLETPRINTAPPLY3	0x0093	Causes the printer to perform print and apply sequence 3
CMD_PALLETPRINTAPPLY4	0x0094	Causes the printer to perform print and apply sequence 4
CMD_PALLETPRESENT	0x0098	Sets the printers' Pallet Present input
CMD_PALLETNOTPRESENT	0x0099	Clears the printers' Pallet Present input
CMD_PARTPALLETAUTOAPPLY	0x009A	Sets the pallet as a part pallet that will be applied by the applicator as normal.
CMD_PARTPALLETMANUALAPPLY	0x009B	Sets the pallet as a part pallet that will have labels applied by the operator.
CMD_PALLETABORT	0x009C	Aborts/Ends the current pallet and updates the time, dates and (SSCC) counts of the next pallet.
CMD_FORCEMICROPURGE	0x0100	Attempt to perform a micro purge. This command is only valid if the machine is offline or online and supports micro purging.

CMD_OPEN_CAPS	0x0201	Causes printers which implement this command to open the cap covering the print head.
CMD_CLOSE_CAPS	0x0202	Causes printers which implement this command to close the cap covering the print head.
CMD_BLOCK_WARM	0x0203	Causes printers which implement this command to enable warming of the print head.

Notes

The 9550 has a set of specific set of commands for triggering print and applies and controlling the update of the counters and SSCC on the pallet label. If the pallet present input is not used each time that Print and Apply sequence 1 is trigger for a pallet the counters will be updated (assuming that the image has previously be printed). To manually control the update of counters on the image the pallet present must be used. It is required that this input is set high (1) before the first print and apply is performed on the pallet, and then set low (0) after the last apply is performed on the pallet which will cause the counters to be updated.

An example sequence for a pallet that requires a two position apply would be:

1. Pallet Present is set high (1) when it arrives at the first apply position.
2. Pallet Print Apply sequence 1 is triggered (the BUSY output will go high as the sequence starts).
3. When the BUSY output goes low the pallet is moved to apply position 2.
4. Pallet Print Apply sequence 2 is triggered (BUSY goes high).
5. When the BUSY output goes low the pallet is released and the Pallet Present is set low (0).

For part pallets, the part pallet command must be sent before any pallet present is sent and before the first print and apply sequence is sent for the pallet. The part pallet will be automatically reset at the end of the next pallet.

Sample Message

The following byte stream contains a Print Command.

```
HOST> 0x05 0x50 0x08 0x00 0x00 0x00 0x01 0x00
```

Delete File (DEL – 0x500B)

This message allows the Host to delete a file from the Zipher printers' internal database. The message is a Host to Zipher printer only message.

The printer will respond with a NUL message if the file is successfully deleted. If the file cannot be found an FNF message will be returned. If the delete command fails for any other reason an ERR message will be returned stating the reason.

The format of this message is as follows:

Name	Size	Description
Message ID	2 Bytes	0x500B
Message Length	4 Bytes	Variable length > 10
File Type	2 Bytes	A value that identifies the type of file that is to be deleted.
File Name String Length	2 Bytes	The length of the file name string, including a NULL terminator and any pad character to make the length even.
File Name String	n Bytes	The name of the file requested. This will not contain any path information but may contain an extension.

The file types that may be deleted are:

Value	Description	Default Extension
0x0001	CIFF File	.CIFF
0x0002	CIFF Data File	.XML
0x0003	Bitmap Graphic	.BMP
0x0004	True Type Font	.TTF
0x0006	Log File	.TXT
0x000A	Template CIFF File	.TCIFF
0x000B	Comma Separated File	.CSV

The filename passed does not require an extension, and if one is not present, the Zipher Printer will append the default one listed above. If an extension is provided, it should match the one in the table.

Sample Message

The following byte stream contains a Delete File message for a file called Strawberry.xml of type Ciff File.

```
HOST> 0x0B 0x50 0x1A 0x00 0x00 0x00 0x01 0x00 0x10 0x00 0x53 0x74 0x72 0x61
      0x77 0x62 0x65 0x72 0x72 0x79 0x2E 0x78 0x6D 0x6C 0x00 0x00
```

Delete File Unicode (DLU – 0x5031)

This message is the same as the DEL message except that the file name is specified using Unicode characters. This message is available in protocol version 4 or later.

The format of this message is as follows:

Name	Size	Description
Message ID	2 Bytes	0x5031
Message Length	4 Bytes	Variable length > 12
File Type	2 Bytes	A value that identifies the type of file that is requested.
File Name String Length	4 Bytes	The length of the file name string in bytes, including a NULL terminator.
File Name String	n Bytes	The name of the file requested as a Unicode string. This will not contain any path information but may contain an extension.

Please refer to the documentation for the DEL message for details of this message.

Delete All Jobs (DEL – 0x500B)

This message allows the Host to delete all jobs from the Zipher printer's internal database. The message is a Host to Zipher printer only message.

The printer will respond with a NUL message if all jobs were successfully deleted. The format of the message is the same as for the **DEL** command except that the File type must be ".CIFF" and the File name string must be an '*' without an extension.

Sample Message

The following byte stream contains a Delete All Jobs message:

```
HOST> 0B-50-0C-00-00-00-01-00-02-00-2A-00
```

Protocol Error Message (ERR – 0x500D)

This message is used by the Zipher Printer to indicate the failure of the previous message it was sent by the host. The message is Zipher Printer to Host only.

The receipt of this message indicates that the previous message that was sent to the printer failed. The Reason Value attempts to provide more information as to the reason for failure.

The format of this message is as follows:

Name	Size	Description
Message ID	2 Bytes	0x5005
Message Length	4 Bytes	8
Reason for Failure	2 Bytes	ID of Failure reason

The defined Reasons for Failure are:

Name	Value	Description
RFF_FAIL	0x0001	No additional information for the failure can be provided.
RFF_DISKSPACE	0x0002	The printer does not have sufficient space remaining to store the file.
RFF_SELECTLOCKED	0x0003	Job Select Failed, a locked print limit is in use
RFF_SELECTBARRED	0x0004	Job Select Failed, binary communications select job is barred by machine mode
RFF_UPDATELOCKED	0x0005	Update Job Data Failed, a locked print limit is in use
RFF_UPDATEBARRED	0x0006	Update Job Data Failed, binary communications update job data is barred by machine mode
RFF_INVALIDCOMMAND	0x0007	Command Failed, the command ID is invalid
RFF_PRNOFFLINE	0x0008	Print Command Failed, the machine is not online
RFF_PRNNOJOB	0x0009	Print Command Failed, no job is selected
RFF_PRNLIMITREACHED	0x000A	Print Command Failed, the print limit has been reached
RFF_PRNBARRED	0x000B	Print Command Failed, binary communication print command has been barred by the printer mode
RFF_NOLOCK	0x000C	Clear Print Limit Lock failed, no lock is in use
RFF_DELFILEINUSE	0x000D	Delete File Failed, the file is in use
RFF_INVALIDFEATURE	0x000E	Lock Feature Failed, the feature ID is invalid
RFF_INVALIDFILESPEC	0x000F	FRQ, FTX failed the file name, type combination were invalid
RFF_FEATUREBARRED	0x0010	Lock Feature Failed, binary communications is barred from accessing this feature
RFF_INVALIDTIMEDATE	0x0011	Set clock failed, the time date values were invalid (out of range)
RFF_COMMANDNOTACCEPTED	0x0012	The received command has a known command ID, but the machine is currently in a state in which the command cannot be executed.
RFF_OUTOFSEQUENCEBMX	0x0013	More than one BMX message has been received without an EMX message.
RFF_INVALIDEMX	0x0014	An EMX message was received with no preceding BMX.
RFF_NOFILESTRANSMITTED	0x0015	A BMX-EMX pair was received without any FTX messages.

RFF_INSUFFICIENTFILES	0x0016	The number of files transmitted between BMX-EMX messages did not match the number specified in the BMX.
RFF_OUTOFSEQUENCEFTX	0x0017	The FTX message was received out of sequence.
RFF_BADCIFFDATA	0x0018	The CIFF data is incorrectly formatted
RFF_NOTIMPLEMENTED	0x0019	The message is not implemented in this version of software.
RFF_INVALIDFILETYPE	0x001A	The specified file type is not recognised
RFF_UNSUPPORTEDVERSION	0x001B	The communications version passed in the HCV message is not supported.
RFF_SOMEERRORSNOTCLEARED	0x001C	Some of the errors requested to be cleared by the CEM command could not be cleared.
RFF_NOERRORSCLEARED	0x001D	None of the errors requested to be cleared by the CEM command were cleared.
RFF_NOTALLOWEDFORSLAVE	0x001E	The requested operation was rejected because the printer is in slave mode and the requested operation is not valid for slave printer.
RFF_JOBSELECTIONINPROGRESS	0x001F	Indicates that another SEL message was received whilst the printer was downloading files as a result of a previous SEL message.
RFF_IMAGEUPDATEQUEUEFULL	0x0020	The SEL/JDU message failed because the image update queue was full.
RFF_INVALIDPRODUCTID	0x0021	The parameters downloaded to the printer were rejected because they were not suitable for this printer model.
RFF_HARDWARENOTSUPPORTED	0x0022	The hardware does not support the requested operation.
RFF_NOTINLINESELECTMODE	0x0023	A SEL, SLU, JDU, LSL, LCR message with line operations was received when the coder was not in line selection mode.
RFF_NOTSUPPORTEDINLINESELECTMODE	0x0024	A SEL, SLU or JDU message was received in Line Selection mode without the JF_LINEUPDATE flag set.
RFF_INVALIDLINEID	0x0025	The line number provided is not valid. Applies to SEL, SLU or JDU messages when the line number is greater that the number of lines configured on the coder.
RFF_PRNTIMEOUT	0x0026	Confirm print timed out before starting the print.
RFF_PRNNOPRODUCTREADYINPUT	0x0027	Confirm print failed, there was no product ready input specified
RFF_PRNNOTREADY	0x0028	Print failed, the printer was not ready to print. This is usually because it was still printing a previous image.

Sample Message

The following byte stream contains a Protocol Error message for out of disk space.

```
ZIPHER>  0x0D 0x50 0x08 0x00 0x00 0x00 0x02 0x00
```

Set Time and Date (CLK – 0x500E)

This message is used by the Host to command the Zipher Printer to set the current time and date at the printer. The message is Host to Zipher Printer only.

The printer will respond with a NUL message if the command completes successfully. If the feature is invalid a Protocol Error of Invalid Time/Date will be returned. Other protocol errors could be returned depending upon the state of the printer. See the protocol error response for more information.

The format of this message is as follows:

Name	Size	Description
Message ID	2 Bytes	0x500E
Message Length	4 Bytes	18
Year Value	2 Bytes	The current year value (years from 1000 to 9999 will be accepted).
Month Value	2 Bytes	The current month value (valid months are 1 to 12).
Day Value	2 Bytes	The current day value (values between 1 and 28 will be accepted, values between 29 and 31 are allowed if the month contains this many days).
Hour Value	2 Bytes	The current hour (value between 0 and 23 are accepted)
Minute Value	2 Bytes	The current minute (0-59)
Second Value	2 Bytes	The current second (0-59)

Sample Message

The following byte stream sets the machines time to 14:33:00 on 15 Jan 2001

```
HOST> 0x0E 0x50 0x12 0x00 0x00 0x00 0xD1 0x07 0x01 0x00 0x0F 0x00 0x0E 0x00
      0x21 0x00 0x00 0x00
```

File Transfer Complete (FTC – 0x500F)

This message is transmitted by the receiving machine at the end of a file transfer, or group of file transfers to indicate that it believes it has received all of the files that are required. This is typically used at the end of job downloads to indicate all the file dependencies are present on the machine. This is a printer to host only message. The Job Transfers section gives a wider description of the use of this message and the other file transfer messages.

There is no required response to this message other than the protocol level ACK/NAK. Its purpose is to allow the Host to become aware that the printer has now received all the required files for this transfer.

The format of this message is as follows:

Name	Size	Description
Message ID	2 Bytes	0x500F
Message Length	4 Bytes	6

There is no data section in this message.

Sample Message

The following byte stream contains a File Transfer Complete message.

```
HOST> 0x0F 0x50 0x06 0x00 0x00 0x00
```

Begin Multiple File Transfer (BMX – 0x5010)

This message is sent by the file-transmitting machine to indicate the start of a multiple file transfer block. If the printer requests a set of files using a wild-card (i.e. F*), or if the Host wishes to send a number of job files, then this message must be sent before the first FTX message. The Job Transfers section indicates the expected usage of this command.

All multiple file transfers that begin with a BMX message must end with an EMX (End Multiple File Transfer) message. There is expected to be 1 or more FTX messages between the BMX and EMX messages.

The valid responses to the BMX message are NUL if the message is accepted and ERR if the message is invalid. The ERR message types that may be returned are RFF_DISKSPACE, and RFF_OUTOFSEQUENCEBMX. In both cases, the printer enters or remains in multiple file transfer mode and requires an EMX message to exit it.

During the period between the BMX and EMX messages, the receiver checks each file that is sent to determine whether there are any dependant files required. If there are, these are added to a required files list, with any duplicates removed. When the EMX is received this list is used to set up a series of FRQs for the dependencies, thus removing the need to download the same graphic or font multiple times.

The format of this message is as follows:

Name	Size	Description
Message ID	2 Bytes	0x5010
Message Length	4 Bytes	18 bytes
Number of Files	4 Bytes	Total number of files in the download
Total File Size	8 Bytes	Total file size of all of the files to be downloaded.

The “Number of Files” and “Total File Size” fields are provided as error checking and early warning mechanisms. Passing a value of 0 for either field will disable the relevant checks for the field.

If the “Number of Files” value is greater than 0 then the printer will check the number of files received before the EMX message matches. The EMX message will respond with an ERR-RFF_INSUFFICIENTFILES message.

If the “Total File Size” value is greater than 0 then the printer will check that there is sufficient space to store the files that are about to be transmitted. This avoids downloading a large number of files only to find there is insufficient space for the last one. No checking is done when the EMX is received to verify that the total file size matches.

Sample Message

The following byte stream contains a Begin Multiple File Transfer message for 10 files of a total of 21354 bytes.

```
HOST> 0x10 0x50 0x12 0x00 0x00 0x00 0x0A 0x00 0x00 0x00 0x6A 0x53 0x00 0x00
      0x00 0x00 0x00 0x00
```

End Multiple File Transfer (EMX – 0x5011)

This message is sent by the file-transmitting machine to indicate the end of a multiple file transfer block. It must have been preceded by a Begin Multiple File Transfer message; otherwise a protocol error will be generated.

All multiple file transfers that begin with a BMX (Begin Multiple File Transfer) message must end with an EMX message. There is expected to be 1 or more FTX messages between the BMX and EMX messages.

The valid responses to the EMX message are NUL if the message is accepted and ERR if the message is invalid. The ERR message types that may be returned are RFF_INVALIDEMX, RFF_NOFILESTRANSMITTED and RFF_INSUFFICIENTFILES. In all cases, the printer will leave the multiple file transfer mode and begin requesting any dependant files. On any future Poll for Requests messages.

Note: When the printer receives a BMX message, a 30 second time out is started. This timeout is reset by FTX messages. If the time out expires, then the printer will automatically exit multiple file transfer mode. This means that the Host has a 30 second window to begin each file transfer.

The format of this message is as follows:

Name	Size	Description
Message ID	2 Bytes	0x5011
Message Length	4 Bytes	6 bytes

There is no data section in this message.

Sample Message

The following byte stream contains a End Multiple File Transfer message.

```
HOST> 0x0F 0x50 0x06 0x00 0x00 0x00
```

Host Communication Version (HCV – 0x5020)

This message is used by the Host to declare its binary communications compatibility to the Zipher Printer. This message should be sent at the start of any communications session with the printer and may be sent multiple times to change the communications version if required. Its purpose is to allow the host to instruct the Zipher printer to avoid sending any messages that it does not support. This enables the host to protect itself from having to support additional functionality that may be added as part of a future update to the printer software.

Each time that a change is made to the Zipher Binary Communications specification that could cause incompatibility with host software, the communications version is increased, and the Zipher printer will only send out the new or amended message if the host declares that its communication version supports the new message type.

When the Zipher Printer receives this message, it will respond with an NUL message to indicate acceptance. This message was added at version 2 of the communications specification. If the version of the printer software (available through the CMV tag) is earlier that 2 then the machine will respond with a protocol error (RFF_FAIL). The printer will respond with a protocol error (RFF_UNSUPPORTEDVERSION) if the host declares a software version that is later that the protocol version supported by the printer software.

If the host does not send this message to the Zipher printer then it will assume that the host supports up to revision 2 of the protocol (the version at which this message was added). Any additional messages or data that are documented as a later revision of the protocol will be omitted from the responses, making the printer appear as if it were a version 2 capable machine.

The format of this message is as follows:

Name	Size	Description
Message ID	2 Bytes	0x5020
Message Length	4 Bytes	12
Comms Revision	2 Bytes	The revision of the communications specification.
Feature Bitmask	4 Bytes	Bitmask of the features supported by this version.

The comms revision indicates the revision of the specification that the CLARiTY supports. This message was only introduced at revision 1.09 of the communications specification, which is revision 2 of the communications protocol.

The feature bitmask is used to indicate whether a particular communication/software feature is supported by this installation of CLARiTY.

The bits defined in the Communication Version (CMV) tag which is documented as part of the Information Response (IRP) message.

Sample Message

The following byte stream contains a Host Communications version message for version 2 of the protocol.

```
HOST> 0x20 0x50 0x0C 0x00 0x00 0x00 0x02 0x00 0x07 0x00 0x00 0x00
```

Clear Error Messages (CEM – 0x5021)

This message is used by the Host to clear faults and warnings that are active on the CLARiTY user interface. This message should be used in conjunction with the ERR/LEI tag of the IRP message, which indicate which faults and warnings are raised. This message allows the host to clear all raised faults and warnings that are marked as user clearable, or to clear specific faults and/or warnings. This message is sent by the Host.

The valid responses to the CEM message are either an NUL if all of the specified errors could be cleared, or a ERR message with either the reason RFF_SOMEERRORSNOTCLEARED if at least one error was cleared, or RFF_NOERRORSCLEARED if no errors could be cleared. If the command is used in its clear all faults or warnings mode, the NUL message will only be returned when there are no faults or warnings active.

The CEM message can only clear faults and warnings that have their CanClear attribute set to 1. If this attribute is set to 0, the problem is still present and cannot be cleared.

Name	Size	Description
Message ID	2 Bytes	0x5021
Message Length	4 Bytes	>= 12 Bytes (variable length)
Behaviour Flags	4 Bytes	Defines the behaviour of this command see below.
Number of Error Definitions	2 Bytes	Number of specific errors to clear – for each item there must be an error definition – can be set to 0.
Error Type Flags	4 Bytes	Error Type Flags see below. The Error Type Flags and Error Name String and Length must be repeated for each of the Number of Error Definitions.
Error Name String Length	2 Bytes	The length of the error name string, including a NULL terminator and any pad character required to make the length even.
Error Name String	n Bytes	The name of the error to clear. This should match the Name attribute returned by in the ERR/LEI tag.

The flags defined for the Behaviour Flags are:

Name	Bit Number	Value	Description
CEMF_CLEARALLFAULTS	0	0x00000001	Instructs CLARiTY to attempt to clear all Faults currently active.
CEMF_CLEARALLWARNINGS	1	0x00000002	Instructs CLARiTY to attempt to clear all warnings currently active.

The flags defined for the Error Type Flags are:

Name	Bit Number	Value	Description
ETF_ISAFAULT	0	0x00000001	Indicates that the Error Name String refers to a Fault (if the bit is set) rather than a warning.

CLARiTY supports all combinations of the “Behaviour Flags” with error definitions. So it is possible to clear all faults and warnings by setting bits 0 and 1 in the Behaviour Flags, and setting the number of error definitions to 0 to clear all user clearable faults in the system. Equally valid is to set the CEMF_CLEARALLFAULTS bit, and to provide a number of error definitions to clear specific warnings.

Sample Message

The following byte stream contains a Clear Error Messages message that clears all warnings and the fault ERRINKOUT.

```
HOST> 0x21 0x50 0x18 0x00 0x00 0x00 0x02 0x00 0x00 0x00 0x01 0x00 0x01 0x00
      0x00 0x00 0x0A 0x00 0x45 0x52 0x52 0x49 0x4E 0x4B 0x4F 0x55 0x54 0x00
```

Erase Queue Item (EQI – 0x5037)

This message is Host to Zipher printer only. It is used to remove entries from the job update queue. The id of the queue entry (returned from the SLU/JDU commands when the JF_RETURNUPDATEID flag is used) is used to identify each entry to be erased. The printer will respond with an EQR message, which will list the entries that were *not* erased because they were not found in the queue.

Note that erasing items from a queue that is in use may give unpredictable results. The results returned to the host only indicate which IDs were found in the queue at the time of processing the request. If the printer is printing entries from the queue at the time of processing the erase message, it is possible that an erase message will be accepted because the ID exists in the queue at the time of processing, but that ID is subsequently rendered and printed before the erase is actually attempted. This can happen because the image rendering and printing system has higher priority access to the queue than other components in the system.

Name	Size	Description
Message ID	2 Bytes	0x5037
Message Length	4 Bytes	>= 8 Bytes (variable length)
Number of items to erase	2 Bytes	The number of ids that follow.
Queue Entry ID	4 Bytes	The ID of the update to be erased. This value is repeated for the number of ids specified above.

Erase Queue Item Response (EQR – 0x5038)

This message is Zipher printer to Host only. It is sent in response to an EQI message. The message contains a list of job update queue ids that were not found in the queue when the EQI message was processed. See the EQI message for more details.

Name	Size	Description
Message ID	2 Bytes	0x5038
Message Length	4 Bytes	>= 8 Bytes (variable length)
Number of items to erase	2 Bytes	The number of ids that follow.
Queue Entry ID	4 Bytes	The ID of the update that was not found. This value is repeated for the number of ids specified above.

Line Select (LSL – 0x503B)

This message is Host to Zipher printer only. It is used to select the actively printing line and will only be accepted in line selection mode. In line selection mode the printer is configured with a number of actively printing lines. Each line has either a job associated with it or is configured to allow product to pass unprinted or fault out if product is seen from the line. This mode is intended for production situations where the coder is printing on products that come from many different production lines. Switching the printing image between lines is faster than selecting different jobs as the printer can cache the image information.

The line select message selects a particular line to be the active line for a single print. A further line select message is required for each product to print even if it is from the same line. The printer will respond with a NUL or JUC message if the message is accepted. In order for this message to be accepted, the printer must be in line selection mode otherwise an ERR(RFF_NOTINLINESELECTMODE) will be returned. An ERR(RFF_INVALIDLINEID) will be returned if the line id is invalid. Valid line numbers range from 1 to the maximum number of lines setup at the printer.

Name	Size	Description
Message ID	2 Bytes	0x503A
Message Length	4 Bytes	16 Bytes
Line ID	2 Bytes	The ID of the line to select.
Flags	4 Bytes	Flags that indicate the behaviour of this message.
Product Limit	4 Bytes	Reserved for future use set to zero.

The flags defined for the Flags are:

Name	Bit Number	Value	Description
LSLF_CLEARLINE	0	0x00000001	Indicates that the line selection should be cleared back to no line selected (the value of the Line ID is ignored if this flag is set).
LSLF_PRODLIMIT	1	0x00000002	Indicates that the product limit contains a valid value. Reserved for future use set to zero.
LSLF_RETURNUPDATEID	2	0x00000004	Indicates that the id of the line selection/clear as it is added to the job update queue should be returned to the host. If this flag is included, the printer will confirm successful receipt of the message with JUC instead of NUL.

Sample Message

The following byte stream contains a Line Select message for line 1.

```
HOST> 0x3B 0x50 0x10 0x00 0x00 0x00 0x01 0x00 0x00 0x00 0x00 0x00 0x00 0x00
      0x00 0x00
```

Confirmed Print (CPR – 0x503C)

This message is Host to Zipher printer only. It is used to trigger a print but requires the printer's product ready signal to be active before the print will start. The message takes a timeout parameter that is used as follows.

When the printer receives the Confirmed Print message it checks the status of the product ready signal. If this is active, then the print starts immediately and then printer will respond with a NUL message. If the timeout value is 0, then printer immediately responds with a protocol error (ERR) RFF_PRNTIMEOUT. If the timeout value is greater than 0 then the printer waits for the requested timeout. If the product ready signal activates during this timeout then the print starts, and printer will send out a NUL message. If the timeout is reached, then the printer responds with a protocol error (ERR) RFF_PRNTIMEOUT.

A protocol error (ERR) message can also be returned in a range of other circumstances. RFF_PRNNOJOB will be returned if there is no job selected. RFF_PRNOFFLINE will be returned if the printer is not Running. RFF_PRNNOPRODUCTREADY will be returned if there is no input configured as a Product Ready input. RFF_PRNNOTREADY will be returned if the printer is printing when the product ready input activates. Failure for any other reason will be indicated by an RFF_FAIL protocol error.

Note the NUL message response only indicates that the printer accepted the print command, it does not mean that the print will definitely occur. If there is no image to print (because the image hasn't been updated since the last print and a print limit has been used) then a NUL message may still be returned, but the printer will fault with an image update failure message.

For thermal transfer printers, any print delay/registration will occur after the product ready signal activates but before the print starts. The print delay parameter may be used to help with timing issues if the next image is still being rendered when the print starts.

Name	Size	Description
Message ID	2 Bytes	0x503C
Message Length	4 Bytes	8 Bytes
Timeout	2 Bytes	Timeout in milliseconds (maximum 10000).

Sample Message

The following byte stream contains a Confirmed Print message with a time out of 500ms.

```
HOST> 0x3C 0x50 0x08 0x00 0x00 0x00 0xF4 0x01
```

Serialisation Information Request (SIR – 0x503E)

This request is used to extract various types of serialisation status information from the Zipher Printer. It can be used to extract record counts, next index to be printed, last index printed, maximum number of records and amount of free space. It can also be used to clear the serialisation buffer.

The only valid use of this message is as a request from the Host machine to the Zipher Printer. The only acceptable response is an Information Response (SRP = 0x503F).

The format of this message is as follows:

Name	Size	Description
Message ID	2 Bytes	0x503E
Message Length	4 Bytes	Variable length >= 10
Number of Tags	2 Bytes	Number of Information Tags
Information Tag IDs	2 * n Bytes	Information Tag IDs of the information the host is interested in.

Number of Tags

This is a 2-byte value that indicates the number of information tags that follow this value. The value may be in the range 1 to 65535. Each Tag ID is a request for a specific set of information or action.

Tag IDs

The Tag ID is a 2-byte value that indicates a set of information that the Host is interested in. The complete list of supported information tags, and the data that is returned for them can be found under the Serialisation Information Response message.

Sample Message

The following byte stream requests information tags 1 and 2.

```
HOST> 0x3E 0x50 0x0C 0x00 0x00 0x00 0x02 0x00 0x01 0x00 0x02 0x00
```

Serialisation Information Response (SRP – 0x503F)

This message provides information about the job serialisation buffer on the Zipher Printer. The information that is returned is dependent upon the information requested in the Serialisation Information Request (SIR – 0x503E) message. The message uses a tag-based format to provide multiple blocks of status information, based on the request.

The only valid use of this message is as a response from the Zipher Printer to the Host machine. It can be returned as a response to a Serialisation Information Request.

The format of this message is as follows:

Name	Size	Description
Message ID	2 Bytes	0x503F
Message Length	4 Bytes	Variable length > 12
Number of Tags	2 Bytes	Number of Information Tags
Information Tag ID	2 Bytes	Information Tag IDs of the information the host is interested in.
Information Tag Size	2 Bytes	Size of the data for this tag including ID and length
Information Tag Data	n Bytes	Tag dependant data, size given by the Tag Size – 4, structure of the data is given under each information tag description.

For each tag in the number of tags section there will be an Information Tag ID, and Information Tag Size and a block of tag data. The next tag information will immediately follow the end of the previous tag block.

Serialisation Information Tag IDs

The following Information Tag IDs are supported:

Name	Value	Description
SRC	0x0001	Serialisation Record Count
SNI	0x0002	Serialisation Next Record Index
SLR	0x0003	Serialisation Last Record Index
SGM	0x0004	Serialisation Get Max Records
SFS	0x0005	Serialisation Get Free Space
SCB	0x0006	Serialisation Clear Buffer

The value of the tag numbers indicates the type of data that will be returned. Tags IDs starting with 0 in their most significant nibble return pure binary data. Tags starting with 1 in their most significant nibble will return a XML data block.

NOTE: All tags are guaranteed to be an even length, and all multi-byte words will fall on an even byte boundary. Where necessary, tags will be padded with an extra NULL to ensure an even length. The Tag size will include this extra byte.

NOTE: The sizes quoted are not fixed and may grow in the future. No information will be removed from a tag, but additional information may be appended to the end of the tag block. Any client software should take this into account.

The following section describes the content and format of each information tags.

Serialisation Record Count (SRC – 0x0001)

The serialisation record count tag reports the number of records in the serialisation buffer.

The format of the serialisation record count Information tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x0001
Information Tag Size	2 Bytes	8
Serialisation Record Count	4 Bytes	The serialisation record count.

Serialisation Next Record Index (SNI – 0x0002)

The serialisation Next Index tag reports next index to be printed from the serialisation buffer.

The format of the serialisation next index Information tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x0002
Information Tag Size	2 Bytes	8
Serialisation Next Index	4 Bytes	The record index of the next record to be printed.

Serialisation Last Record Index (SLR – 0x0003)

The serialisation Last Index tag reports the last index which was printed from the serialisation buffer.

The format of the serialisation last index Information tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x0003
Information Tag Size	2 Bytes	8
Serialisation Last Index	4 Bytes	The index of the last record that was printed.

Serialisation Get Maximum Records (SGM – 0x0004)

The serialisation Get Maximum Records tag reports the maximum number of records allowed in the serialisation buffer.

The format of the serialisation get maximum records Information tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x0004
Information Tag Size	2 Bytes	6
Serialisation Max Records	2 Bytes	The maximum number of records allowed in the serialisation buffer.

Serialisation Get Free Space (SFS – 0x0005)

The serialisation Get Free Space tag reports the amount of free space there is in the serialisation buffer.

The format of the serialisation get free space Information tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x0005
Information Tag Size	2 Bytes	8

Serialisation Get Free Space	4 Bytes	The amount of free space that is available in the serialisation buffer.

Serialisation Clear Buffer (SCB – 0x0006)

The serialisation Clear Buffer tag clears the serialisation buffer of all its records and then reports the amount of free space after it has done so.

The format of the serialisation clear buffer Information tag is:

Name	Size	Description
Information Tag ID	2 Bytes	0x0006
Information Tag Size	2 Bytes	8
Serialisation Free Space	4 Bytes	The amount of free space that is available in the serialisation buffer.

Serialisation Header Only (SHO – 0x5040)

This message adds a header (field name) for use with the serialisation component.

The only valid use of this message is as a response from the Host machine to the Zipher Printer.

The format of the message is as follows:

Name	Size	Description
Message ID	2 Bytes	0x5040
Message Length	4 Bytes	Variable length >= 10
Number of Fields	2 Bytes	The number of field names that will be supplied in this message. (m)
Field Name 1 Length	2 Bytes	The length of the field name string, including a NULL terminator and any pad character to make the length even.
Field Name 1 String	n Bytes	The field name string
	.	
	.	
	.	
Field Name m Length	2 Bytes	The length of the field name string, including a NULL terminator and any pad character to make the length even.
Field Name m String	n Bytes	The field name string

Sample Message

```
HOST> 0x40 0x50 0x1E 0x00 0x00 0x00 0x02 0x00 0x08 0x00 0x46 0x69 0x65 0x
6C 0x64 0x30 0x30 0x00 0x0A 0x00 0x46 0x69 0x65 0x6C 0x64 0x30 0x31 0x31
0x00 0x00
```

This message will add two headers, and one called “Field00” and another called “Field011”.

Serialisation Data Only (SDO – 0x5041)

This message adds a Data entry for the header supplied with the SHO (0x5040) command.

The only valid use of this message is as a response from the Host machine to the Zipher Printer.

The format of the message is as follows:

Name	Size	Description
Message ID	2 Bytes	0x5041
Message Length	4 Bytes	Variable length >= 10
Number of Data	2 Bytes	The number of field data that will be supplied in this message. (m)
Field Data 1 Length	2 Bytes	The length of the data string, including a NULL terminator and any pad character to make the length even.
Field Data 1 String	n Bytes	The field data string
	.	
	.	
	.	
Field Data m Length	2 Bytes	The length of the field data string, including a NULL terminator and any pad character to make the length even.
Field Data m String	n Bytes	The field data string

Sample Message

```
HOST> 0x41 0x50 0x18 0x00 0x00 0x00 0x02 0x00 0x06 0x00 0x44 0x61 0x74 0x61
0x31 0x00 0x06 0x00 0x44 0x61 0x74 0x61 0x32 0x00
```

This message will add two pieces of data into one record, the first field name will be set to “Data1” and the second will be changed to “Data2”.

Serialisation Indexed Data (SID – 0x5042)

This message adds a Data entry for the header supplied with the SHO (0x5040) with the supplied index used as the entry ID/serial number.

The only valid use of this message is as a response from the Host machine to the Zipher Printer.

The format of the message is as follows:

Name	Size	Description
Message ID	2 Bytes	0x5042
Message Length	4 Bytes	Variable length >= 14
Record Index	4 Bytes	The record serial number or ID
Number of Data	2 Bytes	The number of field data that will be supplied in this message. (m)
Field Data 1 Length	2 Bytes	The length of the data string, including a NULL terminator and any pad character to make the length even.
Field Data 1 String	n Bytes	The field data string
	.	
	.	
	.	
Field Data m Length	2 Bytes	The length of the field data string, including a NULL terminator and any pad character to make the length even.

Field Data m String	n Bytes	The field data string

Sample Message

```
HOST> 0x42 0x50 0x1C 0x00 0x00 0x00 0x01 0x00 0x00 0x00 0x00 0x02 0x00 0x06
0x00 0x44 0x61 0x74 0x61 0x31 0x00 0x06 0x00 0x44 0x61 0x74 0x61 0x32 0x00
```

This message will add two new records, the first with the ID “1” and data “Data1” and the second with the ID “2” and data “Data2”.

Serialisation Header & Data (SHD – 0x5043)

This message adds a header and data together to create a complete record that will be put into the serialisation buffer.

The only valid use of this message is as a response from the Host machine to the Zipher Printer.

The format of the message is as follows:

Name	Size	Description
Message ID	2 Bytes	0x5043
Message Length	4 Bytes	Variable length >= 12
Number of Fields	2 Bytes	The number of fields that will be supplied in this message. (m)
Field Name 1 Length	2 Bytes	The length of the field name string, including a NULL terminator and any pad character to make the length even.
Field Name 1 String	N Bytes	The field name string.
Field Data 1 Length	2 Bytes	The length of the data string, including a NULL terminator and any pad character to make the length even.
Field Data 1 String	n Bytes	The field data string
	.	
	.	
	.	
Field Name m Length	2 Bytes	The length of the data string, including a NULL terminator and any pad character to make the length even.
Field Name m String	n Bytes	The field name string.
Field Data m Length	2 Bytes	The length of the field data string, including a NULL terminator and any pad character to make the length even.
Field Data m String	n Bytes	The field data string

Sample Message

```
HOST> 0x43 0x50 0x2E 0x00 0x00 0x00 0x02 0x00 0x08 0x00 0x46 0x69 0x65 0x
6C 0x64 0x30 0x30 0x00 0x06 0x00 0x44 0x61 0x74 0x61 0x31 0x00 0x0A 0x00
0x46 0x69 0x65 0x6C 0x64 0x30 0x31 0x31 0x00 0x00 0x06 0x00 0x44 0x61 0x74
0x61 0x32 0x00
```

This message will add two new headers with data supplied for each. The first header will be “Field00” with data “Data1” and the second header will be “Field011” with data “Data2”.

Serialisation Set Max Records (SMR – 0x5044)

This message allows you to set the maximum number of records allowed in the serialisation buffer.

The only valid use of this message is as a response from the Host machine to the Zipher Printer.

The format of the message is as follows:

Name	Size	Description
Message ID	2 Bytes	0x5044
Message Length	4 Bytes	8
Maximum number of records	2 Bytes	The maximum number of records allowed in the record buffer.

Sample Message

```
HOST> 0x44 0x50 0x08 0x00 0x00 0x00 0xD0 0x07
```

This message will set the maximum number of records allowed in the serialisation buffer to 2000.

Serialisation Change Field Data (SCF – 0x5045)

This message changes field data of non-serialisation fields in the current job.

The only valid use of this message is as a response from the Host machine to the Zipher Printer.

The format of the message is as follows:

Name	Size	Description
Message ID	2 Bytes	0x5045
Message Length	4 Bytes	Variable length >= 12
Number of Fields	2 Bytes	The number of fields that will be supplied in this message. (m)
Field Name 1 Length	2 Bytes	The length of the field name string, including a NULL terminator and any pad character to make the length even.
Field Name 1 String	N Bytes	The field name string.
Field Data 1 Length	2 Bytes	The length of the data string, including a NULL terminator and any pad character to make the length even.
Field Data 1 String	n Bytes	The field data string
	.	
	.	
	.	
Field Name m Length	2 Bytes	The length of the data string, including a NULL terminator and any pad character to make the length even.
Field Name m String	n Bytes	The field name string.
Field Data m Length	2 Bytes	The length of the field data string, including a NULL terminator and any pad character to make the length even.
Field Data m String	n Bytes	The field data string

Sample Message

```
HOST> 0x45 0x50 0x2E 0x00 0x00 0x00 0x02 0x00 0x08 0x00 0x46 0x69 0x65 0x
6C 0x64 0x30 0x30 0x00 0x06 0x00 0x44 0x61 0x74 0x61 0x31 0x00 0x0A 0x00
0x46 0x69 0x65 0x6C 0x64 0x30 0x31 0x31 0x00 0x00 0x06 0x00 0x44 0x61 0x74
0x61 0x32 0x00
```

This message will change the contents of field “Field00” to “Data1” and change the contents of field “Field00” to “Data2” if they are not currently be used as serialisation fields.

Common Operations

This section outlines the sequence of messages that are used to achieve common operations when communicating with Zipher equipment. It details the typical sequence of messages required to achieve the desired results. It also highlights areas that special attention needs to be given to ensure that unexpected side effects are avoided.

Standard Polling Loop

A standard polling loop is used to keep up to date with the status and requirements of the printer. This allows the host application to present status on the current state of the print, to serve the printer with any files it needs, and to instruct the printer on the version of the communications standard that it supports.

When the connection is first established (at boot of the host application, or after the printer has been absent for a period of time) it is usual to enquire the version of the communication specification that the printer supports, and to instruct the printer which version the host application supports.

Host	Message	Printer
On initial connection	Information Request with CMV tag (IRQ) →	[Printer returns its communications version]
	← Information Response (IRP)	Returns information
[If the CMV contains version 2 of later] Host instructs printer on its version	Host Communication Version (HCV) →	[Printer configures its responses to meet the revision of the specification provided]
	← No Data to Report (NUL)	Acknowledge Receipt

Once the negotiation of the version of the communications specification is complete, the standard polling loop can start. This usually consists of alternating IRQ and PFR messages, to request the status of the printer, and then to allow the printer to request any files it needs. The content of the IRQ message is application dependant but typically contains STS and CTS tags. It is also possible to integrate the HCV message into the polling loop to simplify implementation.

A typical polling loop would behave something like:

Host	Message	Printer
Standard Polling Loop (step 1)	Information Request with tags (IRQ) →	[Printer returns request information]
	← Information Response (IRP)	Returns information
Standard Polling Loop (step 2)	Poll for Requests (PFR) →	[Printer Checks to see if any files are needed]
	← File Request (FRQ) OR ← No Data to Report (NUL)	Response with either an FRQ if files required, or NUL if nothing required.

This loop is repeated until either a file is required (see below for description of file transfers), or the host requires to perform some other operation (e.g. select a new job).

Single Job Downloads

Single Job Downloads can occur in two ways. They can be sent down from the Host machine as a result of a printing operation via a FTX message, or they can be requested from the Zipher printer as a result of a file request or job selection via a FRQ message.

If the printer requests the job file, the FRQ message will be sent out in response to a PFR message as illustrated below.

Host	Message	Printer
Standard Polling Loop	Poll For Requests (PFR) →	[Printer Checks to see if a file is needed]
	← File Request (FRQ)	Requests file

At this point the file transmission messages follow the same message flow as the Host starting the file download.

Host	Message	Printer
Host Sends Job File	File Transmit (FTX) →	
	← No Data to Report (NUL)	Printer Acknowledges correct receipt. And begins checking for dependant files.
Standard Polling Loop	Poll For Requests (PFR) →	[Printer Checks to see if a file is needed]
	← File Request (FRQ)	Requests First Dependant file
Host Sends Dependant File	File Transmit (FTX) →	
	← No Data to Report (NUL)	Printer Acknowledges correct receipt.
Standard Polling Loop	Poll For Requests (PFR) →	[Printer Checks to see if any further files are needed]
	← File Transfer complete (FTC)	Printer Indicates that no further files are required.

This message flow indicates the expected behaviour for a job with one dependant file. Obviously, the FTC message would be sent on the first PFR if there were no dependant files. The host should make no assumption that the FRQ/FTC messages will be received on the first PFR after an FTX message, as there may be other requests pending, or the checks required to determine the dependant files may not have completed.

File Dependencies

At present there are two types of dependencies on a CIFF file, graphic files (BMPs) and font files (TTFs). Each type is handled slightly differently.

For graphic files, Clarity will scan through the CIFF file and generate a list of required bitmaps. For each of these files, it will issue a file request with the Graphic type, and the full file and path name as stated within the CIFF. All bitmaps are assumed to be 2-colour. When Clarity selects the CIFF file, it will automatically strip any path information from the graphic file name, and search for it in its current graphic directory. Bitmaps are always requested, even if they already exist within the printer's database, this allows the graphics to be updated.

For font files, Clarity will scan through the CIFF and generate a list of used fonts. It will compare these font names against those installed on the machine. Any that are not present will be requested by the name used in the CIFF file to reference the font.

Multiple Job Downloads

Multiple Job Downloads can occur in two ways. They can be sent down from the Host machine as a group download operation via multiple FTX messages, or they can be requested from the Zipher printer as a result of a wildcard file request via a FRQ message.

If the printer requests the job files, the FRQ message will be sent out in response to a PFR message as illustrated below.

Host	Message	Printer
Standard Polling Loop	Poll For Requests (PFR) →	[Printer Checks to see if a file is needed]
	← File Request (FRQ)	Requests file [F*]

At this point, the file transmission message follows the same message flow as the Host starting the file download.

Host	Message	Printer
Start Multiple File Transfer Operation	Begin Multiple File Transfer (BMX) →	[Printer verifies the message is valid]
	← No Data to Report (NUL)	Printer Acknowledges correct receipt.
Host Sends Job File	File Transmit (FTX) →	
	← No Data to Report (NUL)	Printer Acknowledges correct receipt. And adds dependant files to list.
Host Sends Job File	File Transmit (FTX) →	
	← No Data to Report (NUL)	Printer Acknowledges correct receipt. And adds dependant files to list.
End Multiple File Transfer Operation	End Multiple File Transfer (EMX) →	[Printer verifies the message is valid, and generates the full list of required files]
	← No Data to Report (NUL)	Printer Acknowledges correct receipt.
Standard Polling Loop	Poll for Requests (PFR) →	[Printer Checks to see if a file is needed]
	← File Request (FRQ)	Requests First Dependant file
Host Sends Dependant File	File Transmit (FTX) →	
	← No Data to Report (NUL)	Printer Acknowledges correct receipt.
Standard Polling Loop	Poll for Requests (PFR) →	[Printer Checks to see if any further files are needed]
	← File Transfer complete (FTC)	Printer Indicates that no further files are required.

Using the AIU and AIS Messages

The AIU and AIS messages allow you to configure the printer to send out a message to the host whenever a particular event occurs. This removes the need for the host to continually poll the printer to find out the status of the printer.

The host sends an AIS message when it initial connects to the printer. This message contains a number of identifiers to indicate what events the host is interested in. When any of these events occur, the printer will transmit the indicated message to the host. The packet received will have the ASY flag set. The printer will not transmit the message whilst it is receiving and responding to a message transmitted by the host, it will delay transmission until the transaction has been completed.

Whilst this can reduce the complexity of the host software and remove some overhead, there are a number of issues that the host must cater for in order for this feature to work reliably.

Message Collision

Message collision occurs when the printer and the host both attempt to start a transaction at the same point in time. Once the printer transmits an AIS generated message it will not accept a new message from the host until the host has acknowledged receipt of the message. In the event that the host receives a packet from the printer with the ASY packet flag set, when it is expecting an acknowledgement for a packet it has sent, it should transmit an acknowledgement to the printer for the ASY message, and then continue to await an acknowledgement for the original packet.

The correct recovery for packet collision is indicated below:

Host	Packet	Printer
Host transmits File Request (for example)	File Request (FRQ) → ← Information Response(IRP)	Printer simultaneously transmits an IRP for status change with ASY flag set(for example). Printer receives the FRQ but does not respond.
Host Detects ASY flag and responds to printer with an ACK	ACK packet →	Printer closes the AIU generated transaction. Printer processes the FRQ from the host.
Host receives File Transmission	← File Transfer (FTX)	Printer Acknowledges and the communications proceeds as usual.

If the host software does not acknowledge the AIU generated IRP packet then the printer will initiate the standard retry software and will resend the packet up to 5 times before giving up on the transaction.

Using the AIS message over a TCP/IP connection

When communicating with the printer over a TCP/IP link, the printer will close the socket if it is inactive for more than 20 seconds. When using only the AIS message to cause the printer to transmit message as a result of change of status, it is possible for the TCP/IP socket connection to be inactive for more than 20 seconds, and hence the connection will be closed.

There are two solutions to this problem:

1. Whenever the printer closes the socket connection, re-open it immediately.
2. Send regular synchronous messages, for example IRQ with no tags, every 10 seconds to keep the connection open.

Note: This is not a problem with the serial connection, as the connection is kept permanently open.

Random Imaging Applications

Random imaging applications are applications where each image that is printed is different from the last and the data is supplied by a host application. The Zipher Binary Communications protocol is specifically designed to maximise the performance of printer in these applications. This section gives tips on how to achieve maximum performance in these applications.

Tips:

1. Use TCP/IP as the connection means.

2. Negotiate communications revision 8 or later – this sets the TCP/IP TCP_NODELAY option on the printers' end of the socket connection. By default, TCP/IP will delay transmission of small packets for up to 200ms using it's built in Nagling algorithm, for maximum performance this should be turned off and packets should be sent as soon as they are ready. The host application should also set TCP_NODELAY on its end of the socket connection, this can be done with the following call once the socket has been connected:

```
int nOn = 1;
setsockopt(m_socket, IPPROTO_TCP, TCP_NODELAY, (const char*)&nOn,
           sizeof(nOn));
```

3. Make use of the printers built in image update queue. The printer has an image update queue which contains the Job Selections and Job Data Updates. Adding selections and updates to the queue with print limits set on them forces them to stay in the queue until the print limit is reached. Once printing starts for the last print of the item, it is removed from the queue and the next item is imaged and made ready to print. The printer can normally image the next item well before the previous one has finished printing.

   For maximum performance it is recommended to try to keep at least 2 items in the queue where every possible.

4. Avoid polling the printer for status when it is online and printing either use asynchronous messages (AIS subscriptions) or configure the digit outputs of the printer and monitor these. Normally a host application will monitor print cycle starting and completing, online/offline state and the various error conditions.

5. Where possible prefer to use Job Data Update (JDU) message in preference to Selection (SEL) message. Selecting a job will usually take at least twice as long as updating the data in an already selected job. This is due to the time required to load and parse the job file. If a job needs to be selected at the start of the run, but the data to print is not available yet use the JF_EXPIREDPRODLIMIT flag.

A typical random imaging application

A typical random imaging application has two parts to it – the setup phase and the running phase. The setup phase occurs when the printer and the host first connect after power on or a loss of communications due to error. It will usually consist of most of the following operations:

- Negotiate protocol version [IRQ/HCV]
- Setup asynchronous advises [AIS]
- Check the printer state and place it offline if it is online [IRQ-STS, CMD]
- Setup any key printer parameters (print speed etc.) *[optional]*
- Download any job files [FTX] *[optional]*
- Clear the job queue [CMD]
- Select the starting job [SEL]
- Wait for the job to select [AIR - AIS_RENDERING_COMPLETE]
- Clear all errors [CEM]
- Add any additional products to the queue [SEL/JDU]

- Put the printer online [CMD]

When running the operations will consist of:

- Monitoring the digital output and asynchronous advises for errors
- Adding items to the queue ensuring the queue doesn't overflow [SEL/JDU]
- Watching the digital output and asynchronous advises for print complete to know the printer is ready to accept a new print signal
- In Intermittent applications, watching the digital output and asynchronous advises for printer not busy to know that the printer item can be moved.
- Issuing print signals to the printer when the next item is ready to be printed and the printer has indicated that the previous print has completed, either through the digital print input or the print command [CMD]

Appendix A: Differences in message behaviour

As problems and bugs are found, and the binary communications specification evolves to provide greater power and flexibility it becomes necessary to change the behaviour of certain messages to correct deviations from the specification, or to add additional functionality. This could cause the CLARiTY printer to become incompatible with host applications that are relying on the message behaving in a given way.

In order to avoid this problem, the HCV message is provided to force the CLARiTY printer to obey a given revision of the specification. Providing the host software specifies the version of the specification it is designed to work with when it first connects to printer (via the HCV message), compatibility will be maintained.

The table below is provided to indicate the differences in message behaviour between each revision of the binary communications software.

Revision Change	Message Differences
Changes between 2 and 3	*IRP message that return XML:* All IRP messages that return XML strings are now preceded by the XML processing instruction: <?xml version="1.0" encoding="UTF-8" ?> *SEL message:* Prior to version 3 of the communications specification, if the Job was not present in the CLARiTY database, the FRQ message would not be sent to the host on the next poll. Instead CLARiTY would deselect the current job, and show "No Job Selected" on the home screen.
Changes between 3 and 4	*Packet size:* When used over a transport that provides its own packetization (currently only TCP/IP), packets of up to 64k bytes may be sent and received. Host applications may continue to send smaller packets but must ensure that they are able to receive large packets. *New Messages:* The SLU, DLU, FRU and FTU messages were introduced in this revision, being Unicode equivalents of SEL, DEL, FRQ and FTX respectively. The JIU (Job Information Unicode) tag was also added to the IRQ/IRP messages. You should ensure that any host application using protocol version 4 is capable of handling FRU and FTU in response to PFR messages.
Changes between 4.00 and 4.01	*IRP->CMV and HCV messages:* Gain the Log Files and Onboard Image editing flags in the feature bitmask. *FRQ for directory listings:* Flags added for log files and image template directories. *FRQ, FTX, DEL, FRU, FTU, DLU:* Added Ciff Template file type.
Changes between 4 and 5	*Message Changes:* The JIU (Job Information Unicode) tag now returns the job name as a Unicode string – version 4 incorrectly returns it as an ANSI string. *Connection Changes:* Version 5 communication software will reset its communication protocol back to 2 when a socket connection is closed, or when a serial connection is timed out (no packet received in the last 20 seconds). This prevents backward compatibility problems with software that does not support newer protocol versions. It is recommended that host software avoids negotiating version 4 of the protocol as this could lead to other older software not working with the printer.
Changes between 5 and 7	*Message Changes:* The AIS (Asynchronous Information Subscription) message is added. See page 52 for details. The AIR (Asynchronous Information Response) message is added. See Page 55 for details. The AIU message will now respond with an ACK. AIU should no longer be used as it is deprecated in favour of AIS.

Changes between 7 and 8	*Changes:* Negotiating version 8 or later now sets the TCP_NODELAY option on the socket connection. This can remove up to 400ms from a message transaction when set at both ends of a connection. Due to the timing changes this can cause client applications to break hence the communication revision change.
Changes between 8 and 9	*New Messages:* FTD, LSL *Message Changes:* SEL, SLU and JDU messages have new JF_LINEUPDATE flag added. SEL and SLU have the new JF_EXPIREDPRODLIMIT flag.
Changes between 9 and 10	*Message Changes:* CMV Tag of IRP now has FB_SERIAL and FB_TCPIP flags set as appropriate. Added the JF_AUTOCLEARLIMITFAULT to SEL, SLU and JDU message. FNF message now supports an optional error code to indicate why the file could not be found.
Changes between 10 and 11	*Message Changes:* New file type “Temporary CIFF” supported by all file commands. Added JF_TEMPORARYCIFF to job selection commands.